

SQL Advanced Analytics using the Superstore Dataset

Customer Sales Insights using Subqueries, Common Table Expressions (CTEs) and Window Functions

Celebal Technologies Internship

Assignment No-2: Week 3 SQL Advanced Analytics Assignment

Database: superstore_db

Dataset: Sample Superstore Dataset

Author: Snehal A. Bhosale

College: Sanjivani College OF Engineering, Kopargaon 423603

E-mail: snehalbhosale1807@gmail.com

Contents

Sr. No	Topic	Page No.
1.	Introduction	02
2.	Objective	03
3.	Tools and Technologies Used	03
4.	Dataset Overview	03
5.	Project Workflow	04
6.	Database Creation	04
7.	Raw Table Creation and Data Import	05
8.	Data Validation	07
9.	Data Cleaning and Transformation	12
10.	Creation of Business Tables	13

11.	Subqueries	16
12.	Common Table Expressions (CTEs)	23
13.	Window Functions	33
14.	Final Combined Query	44
15.	Customer Sales Insights (Mini Project)	48
16.	Performance Optimization	56
17.	Key Business Insights	57
18.	Challenges Faced	57
19.	Learning Outcomes	58
20.	Conclusion	58
21.	GitHub Repository	58

1. Project Overview:

This report documents the third week of my SQL data engineering internship assignment, where the focus shifted from basic querying to advanced analytical SQL. The exercise is built around the Sample Superstore dataset, a widely used retail transactions dataset that captures customer orders, shipping details, product categories, and profitability figures across a multi-year period. Rather than treating the dataset as a single flat file, the assignment required me to design a small layered data pipeline inside MySQL Workbench: importing the raw data, validating and cleaning it, restructuring it into normalized business tables, and then running progressively more advanced queries — subqueries, common table expressions, and window functions — to answer real customer-sales questions.

The purpose of this documentation is to walk through every stage of that process in the order it was actually built, explain the reasoning behind each script, and present the SQL logic alongside the business questions it was written to answer.

2. Objective:

- Set up a dedicated MySQL database and import the Sample Superstore dataset into a staging table.
- Validate the imported data for completeness, duplicates, and type correctness before using it for analysis.
- Clean and transform raw fields — particularly the date columns — into proper, query-friendly data types.
- Break the single flat dataset into normalized, business-ready tables for customers, orders, and products.
- Apply subqueries (plain, correlated, and nested) to answer comparative business questions.
- Use Common Table Expressions to simplify multi-step aggregations and make queries easier to read and maintain.
- Apply window functions such as RANK(), DENSE_RANK(), ROW_NUMBER(), LAG(), NTILE(), and SUM() OVER() for ranking and trend analysis.
- Combine joins, CTEs, and window functions into a single consolidated report.
- Produce a customer sales insights mini-project answering ten specific business questions.

3. Tools and Technologies Used:

Tool / Technology	Purpose
MySQL Workbench	Writing, executing, and organizing all SQL scripts
MySQL (Relational Database)	Storing and querying the Superstore dataset
Sample Superstore Dataset (CSV)	Source data for the entire analysis
GitHub	Version control and script hosting
Microsoft Word	Preparing this documentation

4. Database Overview:

The Sample Superstore dataset is a transaction-level retail sales dataset. Each row in the original CSV represents a single line item within an order, and carries details about the customer who placed it, the product purchased, when it was ordered and shipped, and the resulting sales, quantity, discount, and profit figures. The main fields used throughout this project are listed below.

Field Group	Columns
Order Details	Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode
Customer Details	Customer_ID, Customer_Name, Segment
Location Details	Country, City, State, Postal_Code, Region

Product Details	Product_ID, Category, Sub_Category, Product_Name
Transaction Metrics	Sales, Quantity, Discount, Profit

5. Project Workflow:

The project followed a layered, bronze-silver-gold style workflow commonly used in data engineering:

- Bronze Layer: raw CSV data loaded as-is into `superstore_raw`, with dates kept as text to guarantee a safe import.
- Validation: the raw table checked for row counts, nulls, duplicate order IDs, and basic sales statistics.
- Silver Layer: a cleaned table, `superstore_clean`, created with proper DATE columns and corrected data types.
- Gold Layer: three normalized business tables — customers, orders, and products — built from the clean table using `SELECT DISTINCT`.
- Analysis Layer: subqueries, CTEs, window functions, and a final combined query built on top of the gold layer tables.
- Insight Layer: a dedicated mini-project script answering specific customer sales questions.

6. Database Creation:

A dedicated database, `superstore_db`, was created to keep this project isolated from any other schema on the server. The script also drops any pre-existing copy first, so the setup script can be re-run safely during development.

Database Setup

SQL Query:

```
DROP DATABASE IF EXISTS superstore_db;

CREATE DATABASE superstore_db;

USE superstore_db;

SELECT DATABASE() AS current_database;
```

Explanation: Dropping the database first makes the script idempotent — it can be executed repeatedly without manual cleanup. The final `SELECT DATABASE()` call is a simple sanity check confirming that the session is now pointed at `superstore_db` before any tables are created.

7. Raw Table Creation and Data Import:

The staging table, `superstore_raw`, mirrors the CSV file column-for-column. Two deliberate design choices were made at this stage: dates are stored as `VARCHAR` rather than `DATE`, and `Postal_Code` is stored as text.

Bronze Table Definition

SQL Query:

```
CREATE TABLE superstore_raw
(
  Row_ID INT,
  Order_ID VARCHAR(30),
  Order_Date VARCHAR(20),
  Ship_Date VARCHAR(20),
  Ship_Mode VARCHAR(50),
  Customer_ID VARCHAR(30),
  Customer_Name VARCHAR(100),
  Segment VARCHAR(30),
  Country VARCHAR(50),
  City VARCHAR(50),
  State VARCHAR(50),
  Postal_Code VARCHAR(20),
  Region VARCHAR(30),
  Product_ID VARCHAR(30),
  Category VARCHAR(50),
  Sub_Category VARCHAR(50),
  Product_Name VARCHAR(255),
  Sales DECIMAL(10,2),
```

```

Quantity INT,

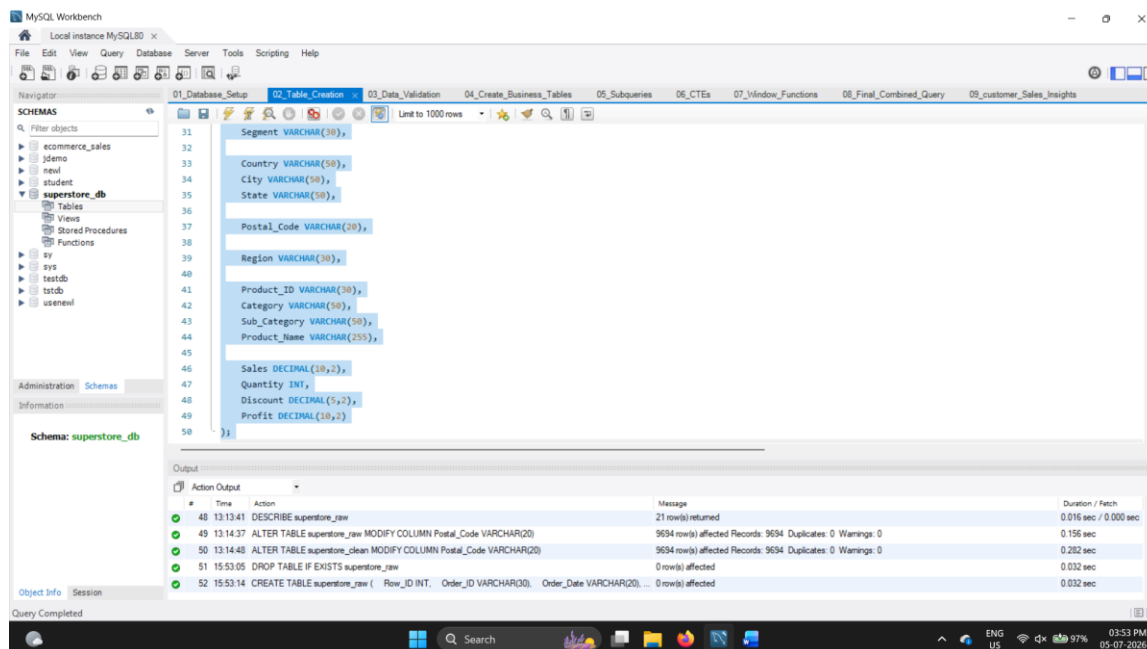
Discount DECIMAL(5,2),

Profit DECIMAL(10,2)

);

```

Output:



Explanation: Storing Order_Date and Ship_Date as VARCHAR at this stage avoids import failures that occur when MySQL tries to coerce US-style MM/DD/YYYY text directly into a DATE column. Postal_Code is kept as VARCHAR rather than INT because postal codes are identifiers, not numeric quantities — leading zeros would otherwise be silently dropped.

The CSV file itself was loaded using MySQL Workbench's Table Data Import Wizard (right-click on superstore_raw → Table Data Import Wizard), rather than a scripted LOAD DATA statement, since the wizard handles column mapping and encoding detection interactively.

Post-import correction: after loading, the Postal_Code column type was revisited to make sure it remained text on both the raw and cleaned tables.

Postal Code Correction

SQL Query:

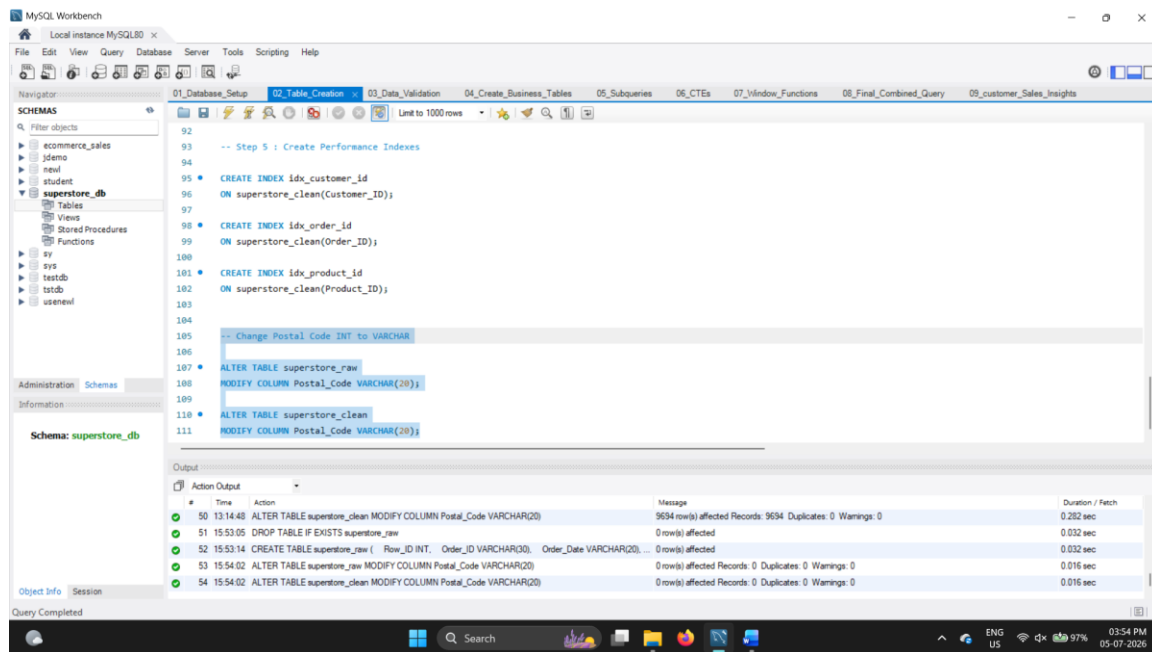
```
ALTER TABLE superstore_raw
```

```
MODIFY COLUMN Postal_Code VARCHAR(20);
```

```
ALTER TABLE superstore_clean
```

```
MODIFY COLUMN Postal_Code VARCHAR(20);
```

Output:



Explanation: This ALTER statement was applied to both the raw and cleaned tables to keep their schemas consistent.

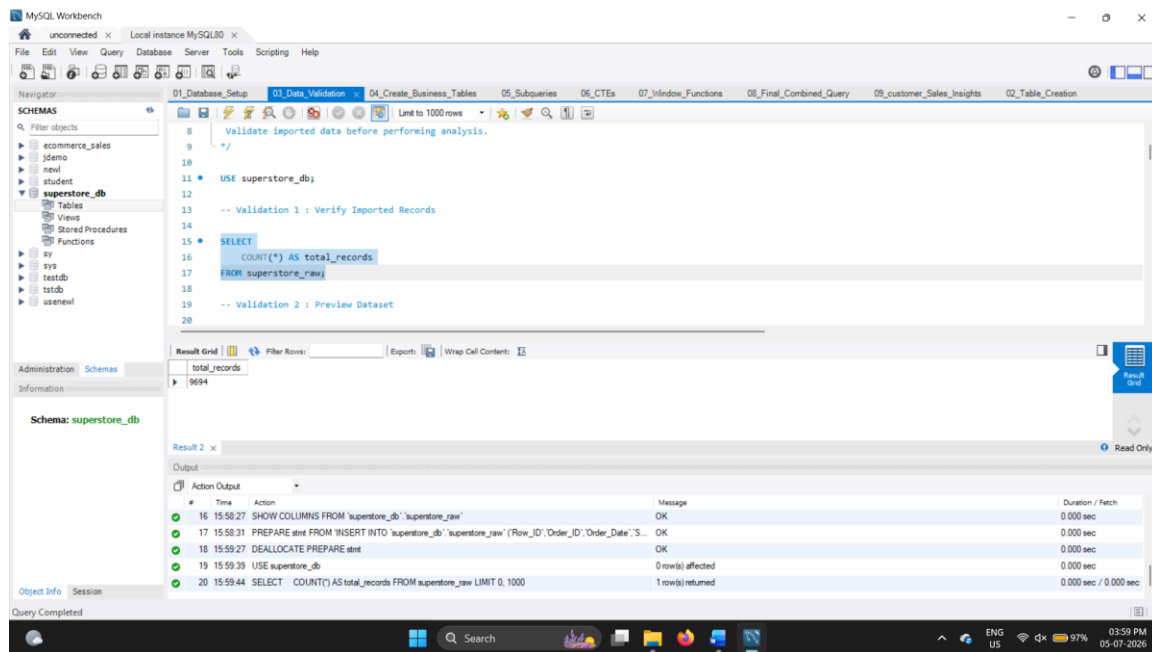
8. Data Validation:

Before any analysis was performed, the imported dataset was checked from several angles — record counts, structure, null values, duplicates, and basic statistics — to confirm the import had succeeded cleanly.

Record Count Check

SQL Query:

```
SELECT COUNT(*) AS total_records FROM superstore_raw;
```



Explanation: Confirms the total number of rows successfully imported into the staging table, which should match the row count of the source CSV file.

Null Value Check

SQL Query:

```

SELECT

    SUM(Customer_ID IS NULL) AS null_customer_id,

    SUM(Customer_Name IS NULL) AS null_customer_name,

    SUM(Order_ID IS NULL) AS null_order_id,

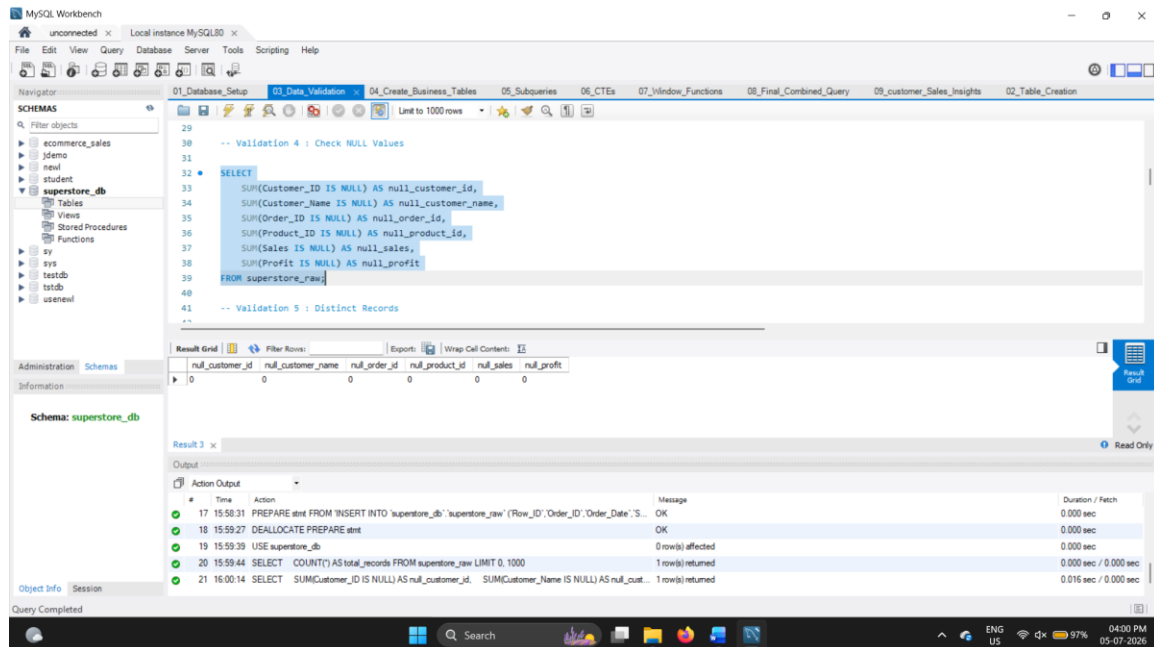
    SUM(Product_ID IS NULL) AS null_product_id,

    SUM(Sales IS NULL) AS null_sales,

    SUM(Profit IS NULL) AS null_profit

FROM superstore_raw;

```

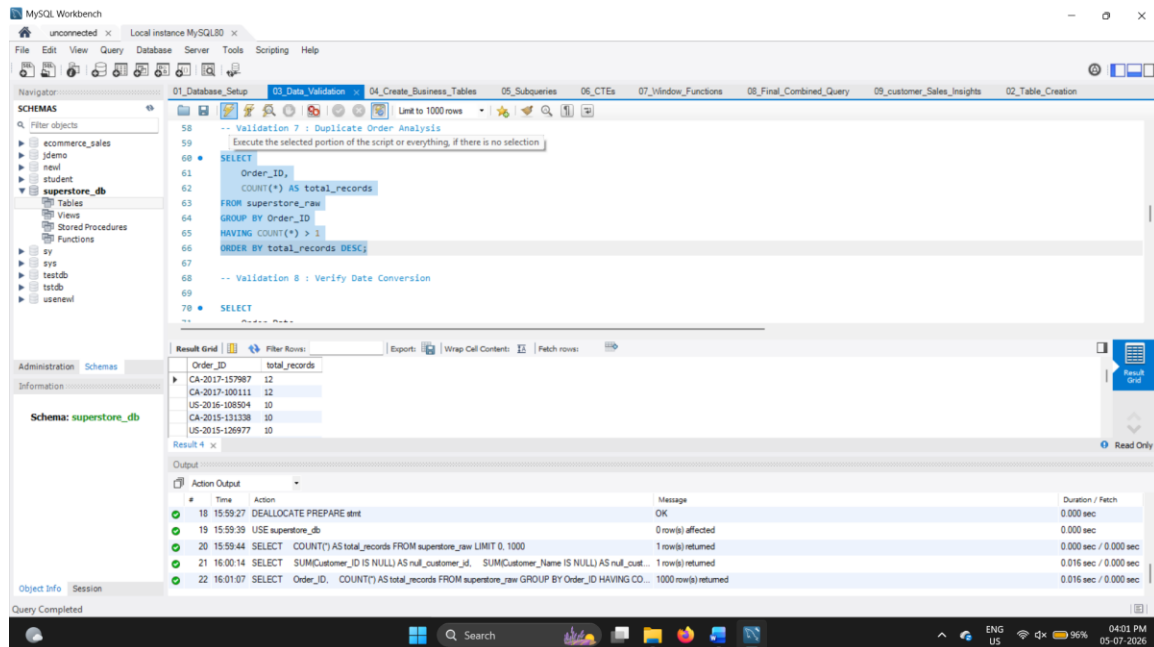



Explanation: SUM() on a boolean expression is a compact way to count NULLs across several key columns in a single pass, rather than running six separate COUNT queries.

Duplicate Order Analysis

SQL Query:

```
SELECT
  Order_ID,
  COUNT(*) AS total_records
FROM superstore_raw
GROUP BY Order_ID
HAVING COUNT(*) > 1
ORDER BY total_records DESC;
```



Explanation: Because a single Order_ID can legitimately appear on multiple rows (one row per line item), this query is used to understand the distribution of multi-item orders rather than to flag genuine duplicate records.

Sales Statistics

SQL Query:

```
SELECT

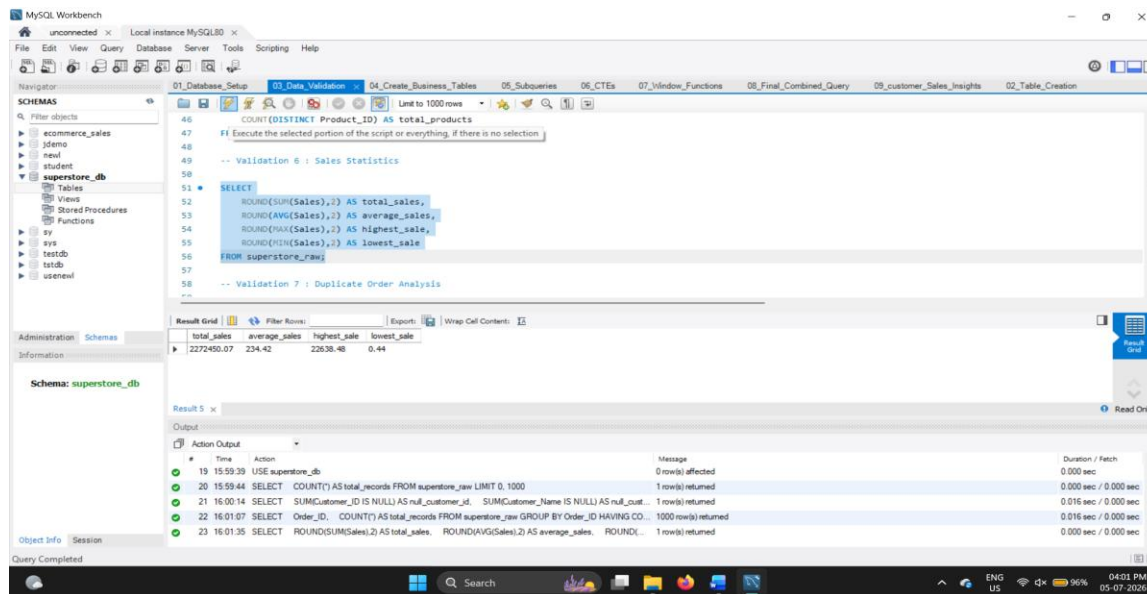
  ROUND(SUM(Sales),2) AS total_sales,

  ROUND(AVG(Sales),2) AS average_sales,

  ROUND(MAX(Sales),2) AS highest_sale,

  ROUND(MIN(Sales),2) AS lowest_sale

FROM superstore_raw;
```



Explanation: A quick statistical summary of the Sales column, used as a baseline reference point before deeper analysis begins.

Raw vs Clean Record Count

SQL Query:

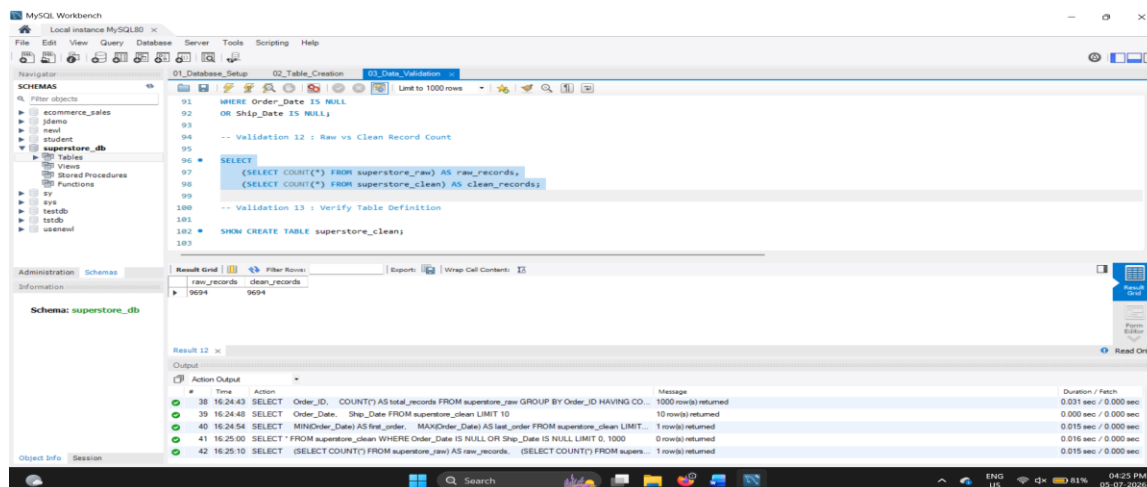
```

SELECT

(SELECT COUNT(*) FROM superstore_raw) AS raw_records,

(SELECT COUNT(*) FROM superstore_clean) AS clean_records;

```



Explanation: Two independent scalar subqueries placed side by side confirm that no rows were lost or duplicated while building the cleaned table.

Additional validation steps executed in the same script included DESCRIBE checks on both tables, SHOW INDEXES, an EXPLAIN plan on a Customer_ID lookup, and SHOW CREATE TABLE to review the final schema definition — all standard housekeeping checks before moving into transformation.

9. Data Cleaning and Transformation

The main transformation applied while building the Silver layer was converting the text-based date columns into genuine DATE values using STR_TO_DATE(), matched against the source format (MM/DD/YYYY).

Cleaned Table Creation

SQL Query:

```
CREATE TABLE superstore_clean AS

SELECT

    Row_ID,

    Order_ID,

    STR_TO_DATE(Order_Date,'%m/%d/%Y') AS Order_Date,

    STR_TO_DATE(Ship_Date,'%m/%d/%Y') AS Ship_Date,

    Ship_Mode,

    Customer_ID,

    Customer_Name,

    Segment,

    Country,

    City,

    State,

    Postal_Code,

    Region,

    Product_ID,
```

```
Category,  
Sub_Category,  
Product_Name,  
Sales,  
Quantity,  
Discount,  
Profit  
FROM superstore_raw;
```

Explanation: CREATE TABLE ... AS SELECT builds the new table and populates it in one statement, applying the date conversion inline. Keeping the raw table untouched means the original import is always available if the cleaning logic ever needs to be revisited.

After the clean table was built, indexes were added on Customer_ID, Order_ID, and Product_ID to speed up the joins used throughout the rest of the project, and a final round of validation (date range checks, an invalid-date scan, and a row-count comparison against the raw table) confirmed the transformation had not introduced errors.

Performance Indexes

SQL Query:

```
CREATE INDEX idx_customer_id ON superstore_clean(Customer_ID);  
  
CREATE INDEX idx_order_id ON superstore_clean(Order_ID);  
  
CREATE INDEX idx_product_id ON superstore_clean(Product_ID);
```

Explanation: These three columns are the ones most frequently used in JOIN and WHERE clauses across the rest of the project, so indexing them upfront benefits every later query rather than any single one.

10. Creation of Business Tables

The cleaned table still mixes customer, order, and product information into one wide structure. To bring the schema closer to a proper relational design, three normalized tables — customers, orders, and products — were created using SELECT DISTINCT against superstore_clean.

Customers Table

SQL Query:

```
CREATE TABLE customers AS  
SELECT DISTINCT  
    Customer_ID,  
    Customer_Name,  
    Segment,  
    Country,  
    City,  
    State,  
    Postal_Code,  
    Region  
FROM superstore_clean;
```

Explanation: SELECT DISTINCT collapses the repeated customer rows down to one row per unique Customer_ID, giving a proper customer dimension table.

Orders Table

SQL Query:

```
CREATE TABLE orders AS  
SELECT DISTINCT  
    Order_ID,  
    Order_Date,  
    Ship_Date,  
    Ship_Mode,  
    Customer_ID,  
    Sales,  
    Quantity,
```

```
Discount,  
  
Profit  
  
FROM superstore_clean;
```

Explanation: The orders table keeps the transactional metrics (Sales, Quantity, Discount, Profit) alongside a foreign key reference to Customer_ID, which is what all of the later subqueries, CTEs, and window functions are built against.

Products Table

SQL Query:

```
CREATE TABLE products AS  
  
SELECT DISTINCT  
  
    Product_ID,  
  
    Category,  
  
    Sub_Category,  
  
    Product_Name  
  
FROM superstore_clean;
```

Explanation: A separate product dimension table, used mainly in category- and sub-category-level reporting.

Indexes were then created on the primary lookup columns of all three tables (Customer_ID, Order_ID, Product_ID), and a relationship check confirmed that customers and orders join correctly on Customer_ID.

Relationship Verification

SQL Query:

```
SELECT  
  
    c.Customer_ID,  
  
    c.Customer_Name,  
  
    o.Order_ID,
```

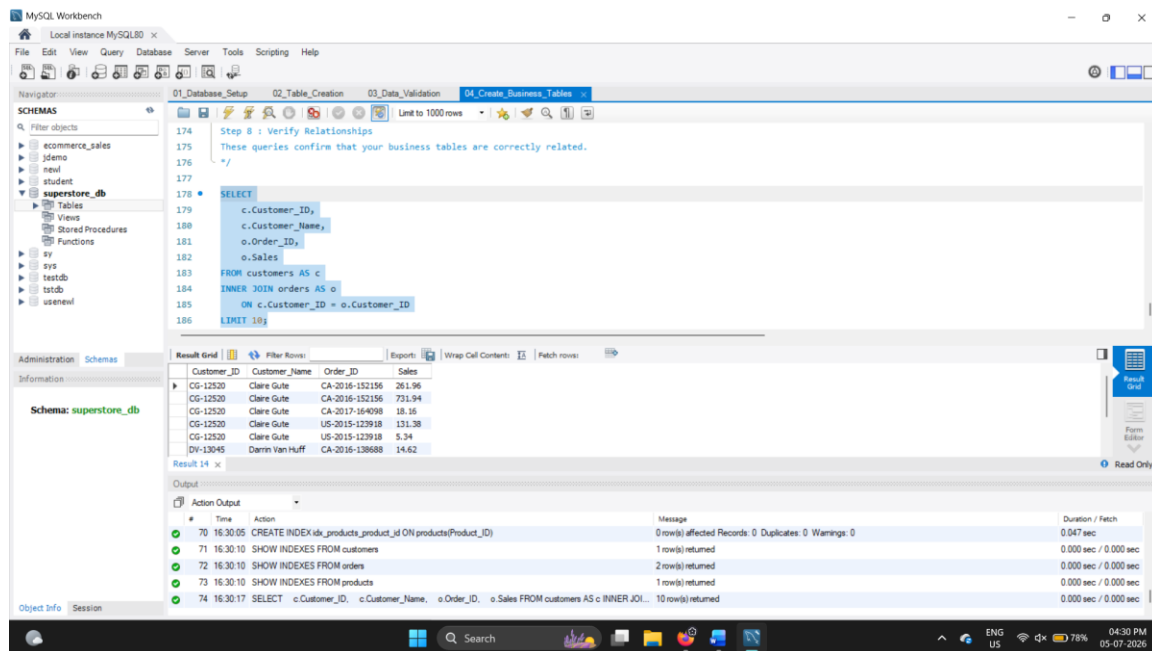
o.Sales

FROM customers AS c

INNER JOIN orders AS o

ON c.Customer_ID = o.Customer_ID

LIMIT 10;



Explanation: A simple INNER JOIN sanity check — if this query returns matched rows for a sample of customers, the foreign key relationship between the two tables is working as intended.

11. Subqueries:

With the business tables in place, the next stage of the assignment focused on subqueries — both plain subqueries evaluated once, and correlated subqueries evaluated once per outer row. Six questions were answered using this technique.

Query 1 — Orders Above Average Sales

Concept Applied: Subquery (non-correlated)

SQL Query:


```

SELECT

    o.Order_ID,

    o.Customer_ID,

    o.Sales,

    o.Quantity,

    o.Profit

FROM orders AS o

WHERE o.Sales >

(

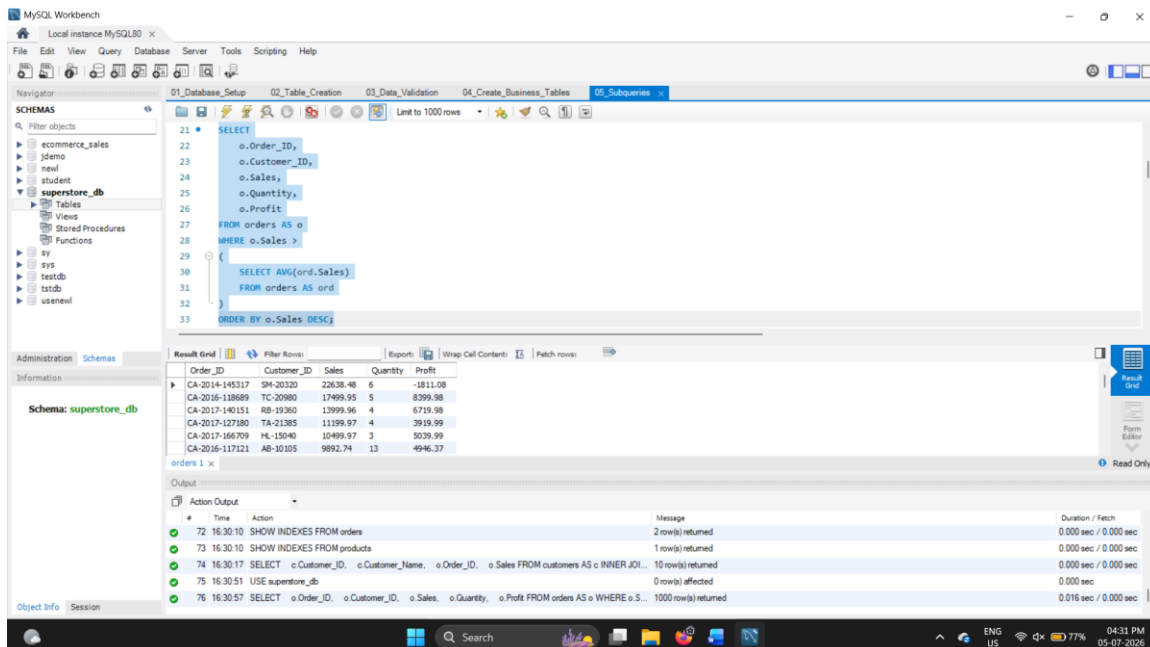
    SELECT AVG(ord.Sales)

    FROM orders AS ord

)

ORDER BY o.Sales DESC;

```



Explanation: The inner query computes a single number — the average sale value across all orders — and the outer query filters for orders exceeding it. This is a classic use of a scalar subquery to flag above-average transactions.

Query 2 — Highest Sale per Customer

Concept Applied: Correlated Subquery

SQL Query:

```
SELECT

    o.Customer_ID,

    o.Order_ID,

    o.Sales,

    o.Profit

FROM orders AS o

WHERE o.Sales =

(

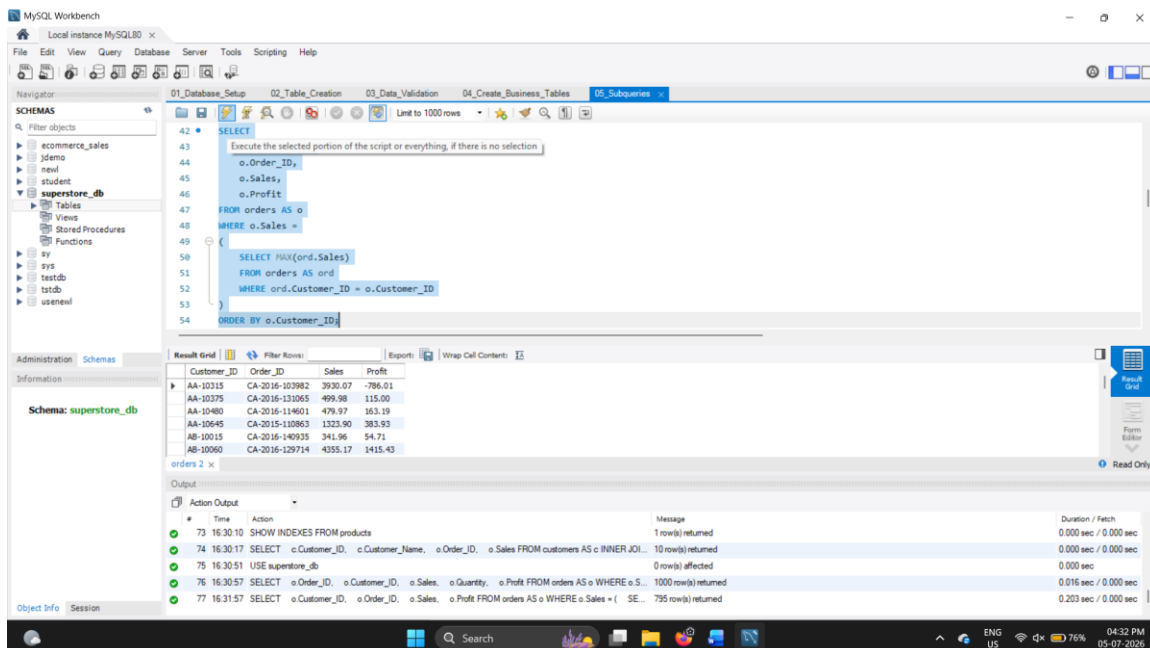
    SELECT MAX(ord.Sales)

    FROM orders AS ord

    WHERE ord.Customer_ID = o.Customer_ID

)

ORDER BY o.Customer_ID;
```



The screenshot displays the MySQL Workbench interface. The SQL Editor contains the following query:

```
SELECT
    o.Order_ID,
    o.Sales,
    o.Profit
FROM orders AS o
WHERE o.Sales =
(
    SELECT MAX(ord.Sales)
    FROM orders AS ord
    WHERE ord.Customer_ID = o.Customer_ID
)
ORDER BY o.Customer_ID;
```

The Results Grid shows the following data:

Customer_ID	Order_ID	Sales	Profit
AA-10315	CA-2016-103982	3930.07	-786.01
AA-10375	CA-2016-133065	499.98	115.00
AA-10480	CA-2016-114601	479.97	163.19
AA-10645	CA-2016-110863	1323.90	383.93
AB-10013	CA-2016-140935	341.96	54.71
AB-10060	CA-2016-129714	4355.17	1415.43

The Output pane shows the execution log with the following entries:

- 73 16:30:10 SHOW INDEXES FROM products 1 row(s) returned 0.000 sec / 0.000 sec
- 74 16:30:17 SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Sales FROM customers AS c INNER JOIN... 10 row(s) returned 0.000 sec / 0.000 sec
- 75 16:30:51 USE superstore_db 0 row(s) affected 0.000 sec
- 76 16:30:57 SELECT o.Order_ID, o.Customer_ID, o.Sales, o.Quantity, o.Profit FROM orders AS o WHERE o.S... 1000 row(s) returned 0.016 sec / 0.000 sec
- 77 16:31:57 SELECT o.Customer_ID, o.Order_ID, o.Sales, o.Profit FROM orders AS o WHERE o.Sales = (SE... 795 row(s) returned 0.203 sec / 0.000 sec

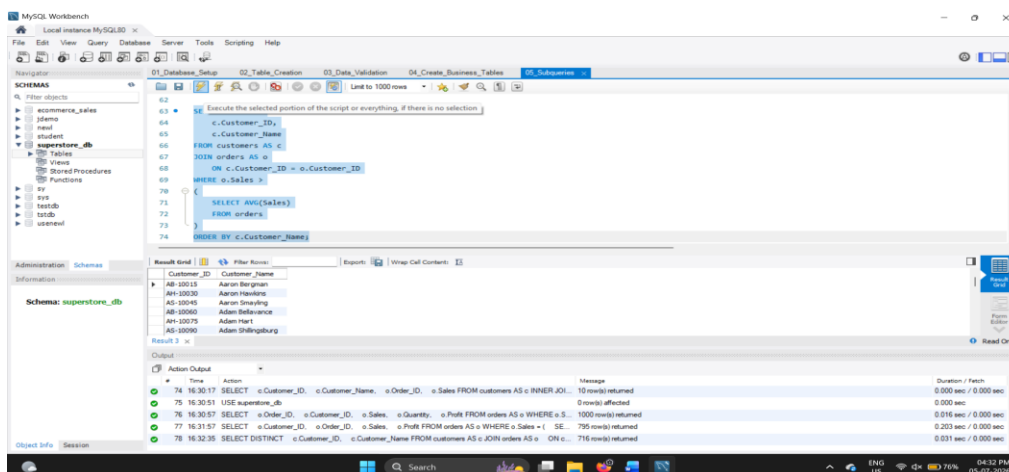
Explanation: This subquery is correlated because it references o.Customer_ID from the outer query, so it re-evaluates MAX(Sales) separately for every customer, rather than once for the whole table.

Query 3 — Customers Above the Overall Average Sale

Concept Applied: Subquery

SQL Query:

```
SELECT DISTINCT
    c.Customer_ID,
    c.Customer_Name
FROM customers AS c
JOIN orders AS o
    ON c.Customer_ID = o.Customer_ID
WHERE o.Sales >
(
    SELECT AVG(Sales)
    FROM orders
)
ORDER BY c.Customer_Name;
```



Explanation: Combines a JOIN with a scalar subquery to identify which named customers have placed at least one order above the store-wide average sale value. DISTINCT prevents a customer from appearing more than once if several of their orders qualify.

Query 4 — Order(s) with the Highest Profit

Concept Applied: Scalar Subquery

SQL Query:

```
SELECT

    Order_ID,

    Customer_ID,

    Sales,

    Profit

FROM orders

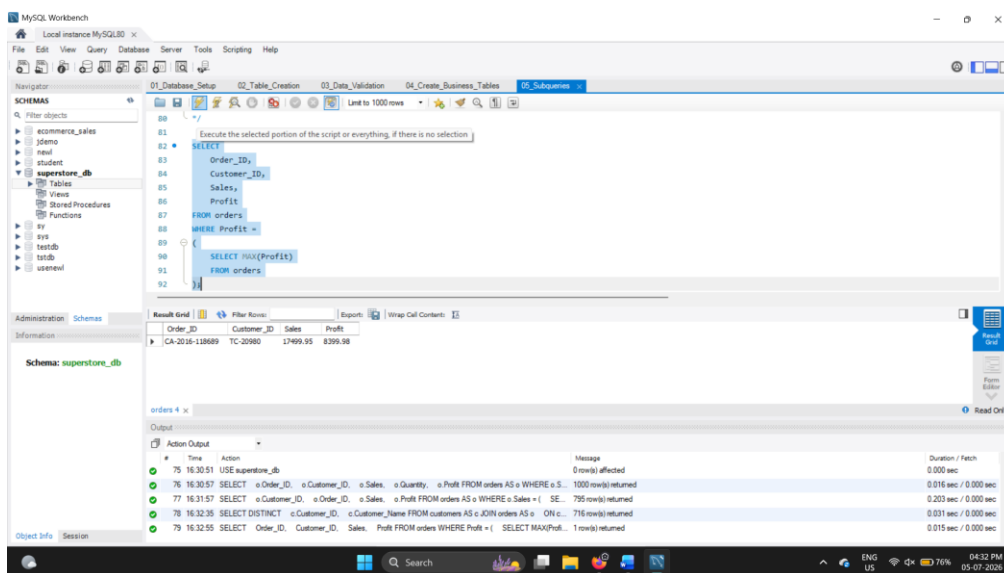
WHERE Profit =

(

    SELECT MAX(Profit)

    FROM orders

);
```



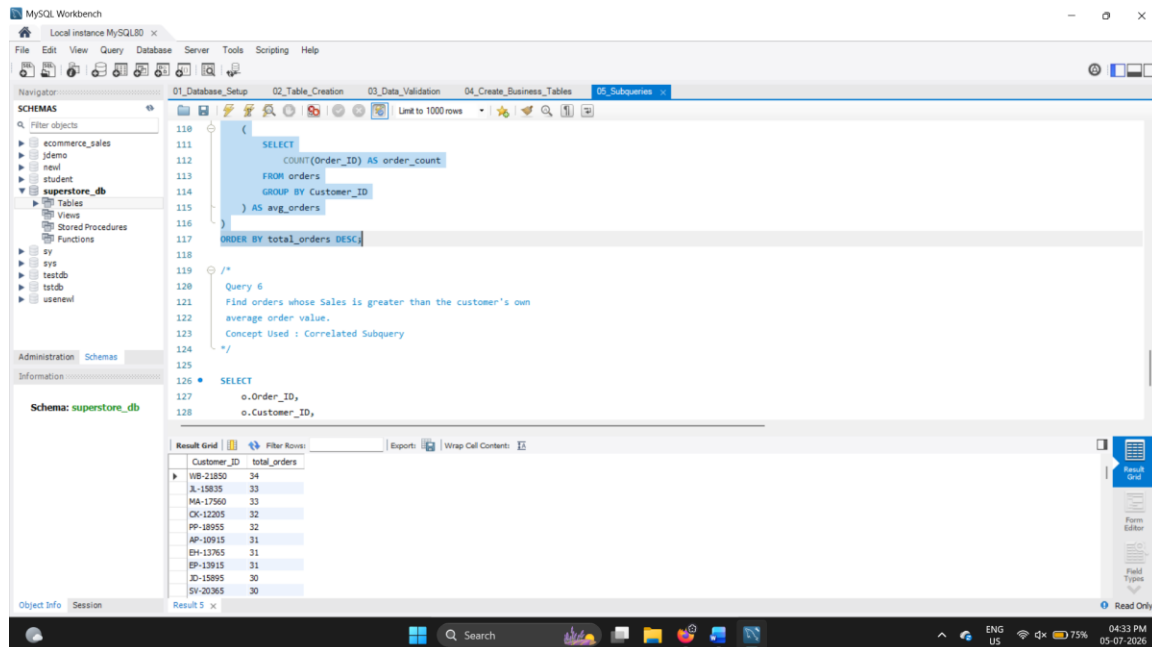
Explanation: A straightforward scalar subquery returning the single largest profit value, used to filter the orders table down to the most profitable transaction(s).

Query 5 — Customers with an Above-Average Number of Orders

Concept Applied: Nested Subquery

SQL Query:

```
SELECT
    Customer_ID,
    COUNT(Order_ID) AS total_orders
FROM orders
GROUP BY Customer_ID
HAVING COUNT(Order_ID) >
(
    SELECT AVG(order_count)
    FROM
    (
        SELECT
            COUNT(Order_ID) AS order_count
        FROM orders
        GROUP BY Customer_ID
    ) AS avg_orders
)
ORDER BY total_orders DESC;
```



Explanation: This is a subquery nested inside another subquery: the innermost query counts orders per customer, the middle query averages those counts, and the outer query's HAVING clause keeps only customers whose own order count beats that average.

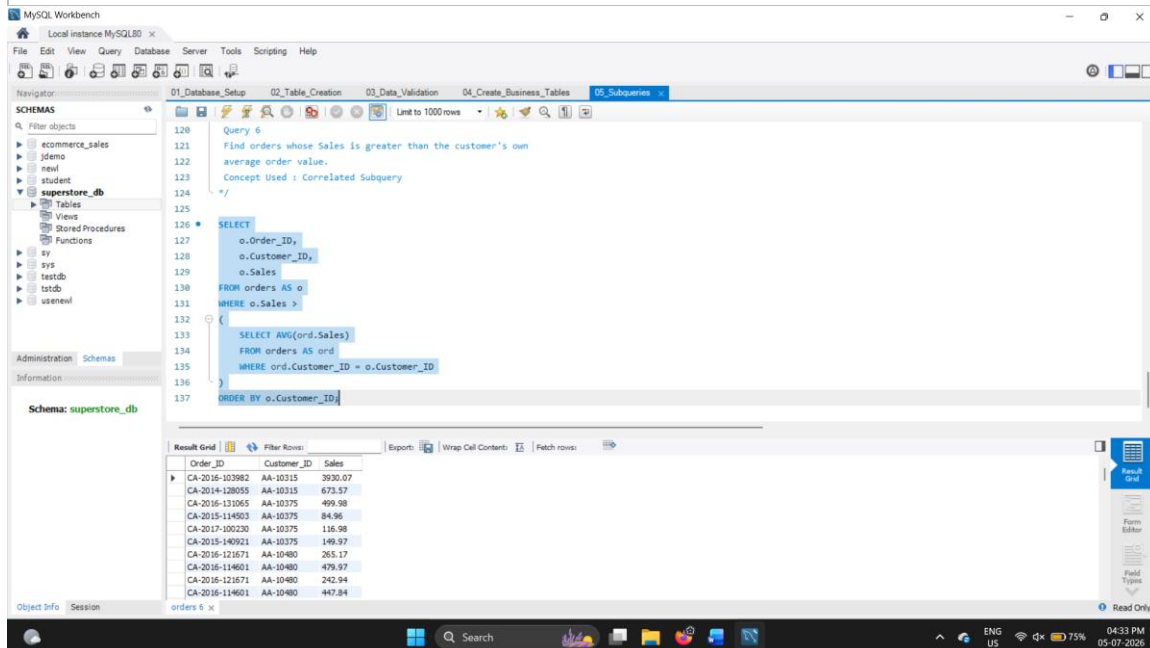
Query 6 — Orders Above a Customer's Own Average

Concept Applied: Correlated Subquery

SQL Query:

```
SELECT
    o.Order_ID,
    o.Customer_ID,
    o.Sales
FROM orders AS o
WHERE o.Sales >
(
    SELECT AVG(ord.Sales)
    FROM orders AS ord
    WHERE ord.Customer_ID = o.Customer_ID
)
```

```
ORDER BY o.Customer_ID;
```



Explanation: Unlike Query 1, this compares each order against that same customer's personal average order value rather than the global average, making it useful for spotting a customer's unusually large purchases relative to their own typical behaviour.

12. Common Table Expressions (CTEs):

Common Table Expressions were used to break multi-step aggregations into named, readable building blocks. Rather than nesting subqueries several levels deep, a WITH clause computes an intermediate result set once and lets the main query reference it by name — improving readability and making the logic easier to reuse and debug.

Query 1 — Total Sales per Customer

SQL Query:

```
WITH customer_sales_summary AS
(
    SELECT
        o.Customer_ID,
        SUM(o.Sales) AS total_sales
```

```

FROM orders AS o

GROUP BY o.Customer_ID
)

SELECT

    cs.Customer_ID,

    c.Customer_Name,

    ROUND(cs.total_sales,2) AS total_sales

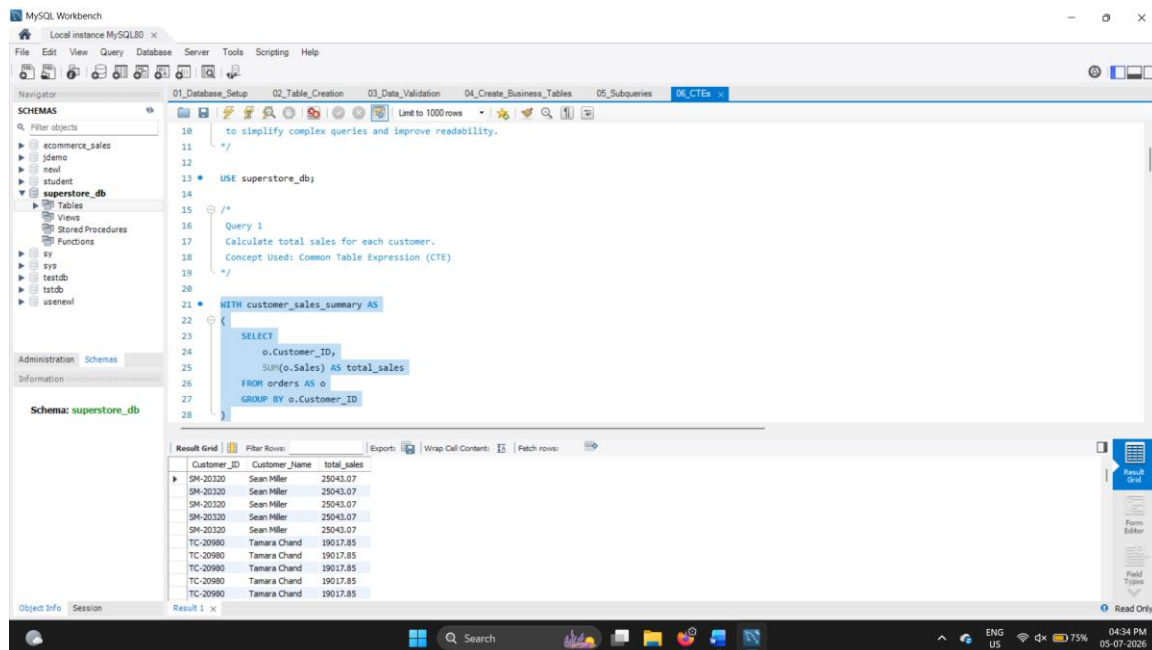
FROM customer_sales_summary AS cs

INNER JOIN customers AS c

ON cs.Customer_ID = c.Customer_ID

ORDER BY total_sales DESC;

```

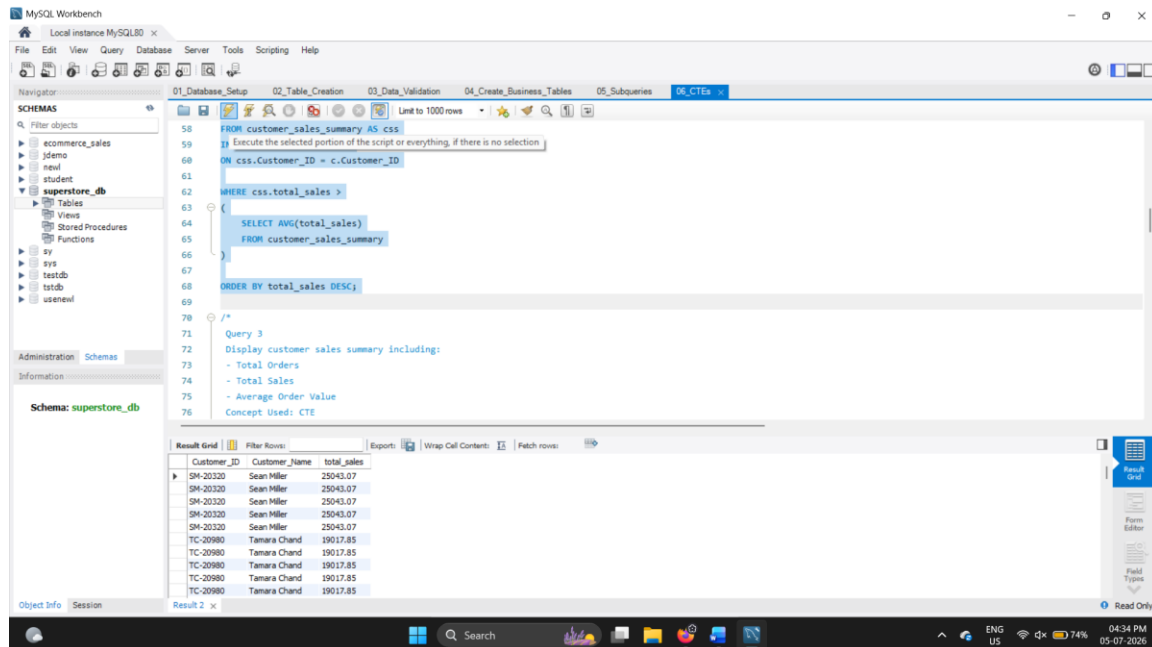


Explanation: The CTE pre-aggregates total sales per customer once, and the outer SELECT simply joins that summary to the customers table to attach readable names — separating the 'aggregate' step from the 'label' step.

Query 2 — Customers Above Average Total Sales

SQL Query:


```
WITH customer_sales_summary AS
(
    SELECT
        Customer_ID,
        SUM(Sales) AS total_sales
    FROM orders
    GROUP BY Customer_ID
)
SELECT
    css.Customer_ID,
    c.Customer_Name,
    ROUND(css.total_sales,2) AS total_sales
FROM customer_sales_summary AS css
INNER JOIN customers AS c
ON css.Customer_ID = c.Customer_ID
WHERE css.total_sales >
(
    SELECT AVG(total_sales)
    FROM customer_sales_summary
)
ORDER BY total_sales DESC;
```



Explanation: This query shows a CTE and a subquery working together: the CTE is defined once but referenced twice — first joined to customers, then reused inside the WHERE clause to compute the average — avoiding writing the SUM(Sales) aggregation logic a second time.

Query 3 — Customer Summary (Orders, Sales, AOV)

SQL Query:

```

WITH customer_summary AS
(
    SELECT
        Customer_ID,
        COUNT(Order_ID) AS total_orders,
        SUM(Sales) AS total_sales,
        AVG(Sales) AS average_order_value
    FROM orders
    GROUP BY Customer_ID
)
SELECT
    cs.Customer_ID,

```

```

c.Customer_Name,

cs.total_orders,

ROUND(cs.total_sales,2) AS total_sales,

ROUND(cs.average_order_value,2) AS average_order_value

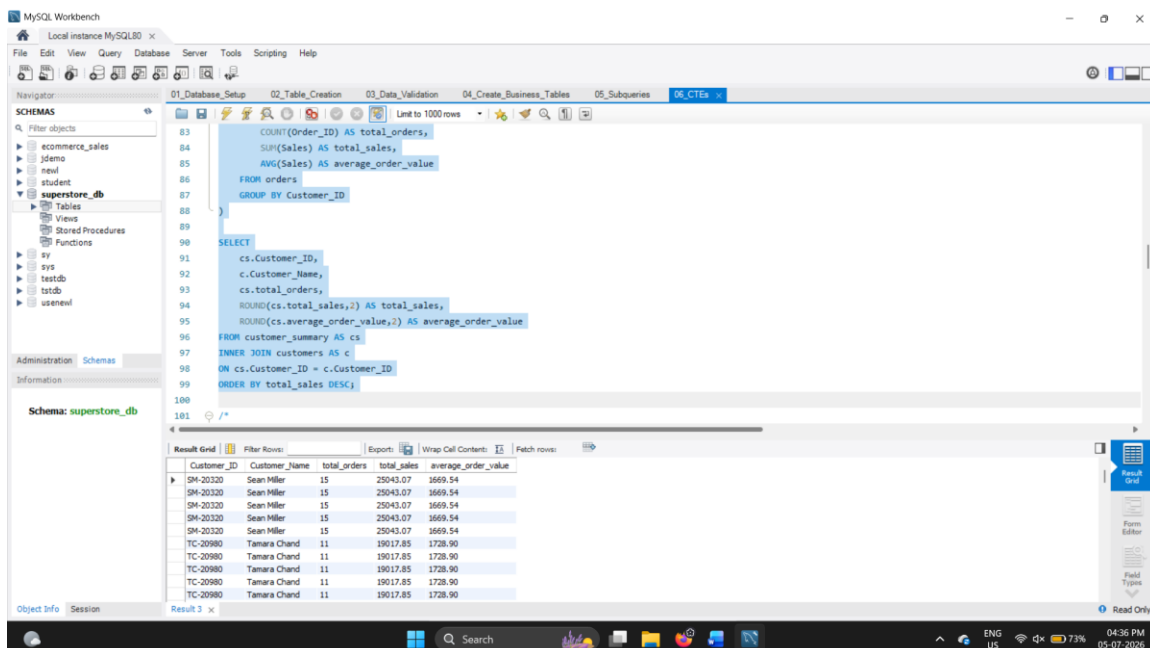
FROM customer_summary AS cs

INNER JOIN customers AS c

ON cs.Customer_ID = c.Customer_ID

ORDER BY total_sales DESC;

```



Explanation: A single CTE computes three related metrics — order count, total sales, and average order value — in one pass over the orders table, which is more efficient than writing three separate aggregate queries.

Query 4 — Customer-wise Profit

SQL Query:

```

WITH customer_profit_summary AS

```

```

(

```

```

SELECT

    Customer_ID,

    SUM(Profit) AS total_profit

FROM orders

GROUP BY Customer_ID

)

SELECT

    cps.Customer_ID,

    c.Customer_Name,

    ROUND(cps.total_profit,2) AS total_profit

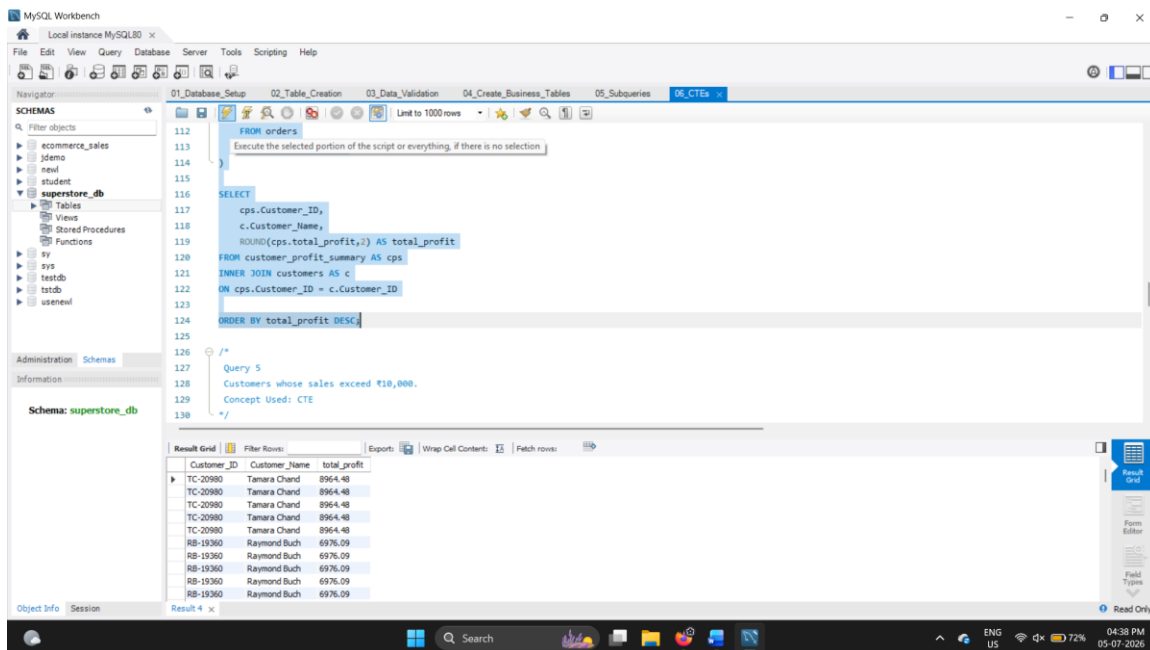
FROM customer_profit_summary AS cps

INNER JOIN customers AS c

ON cps.Customer_ID = c.Customer_ID

ORDER BY total_profit DESC;

```

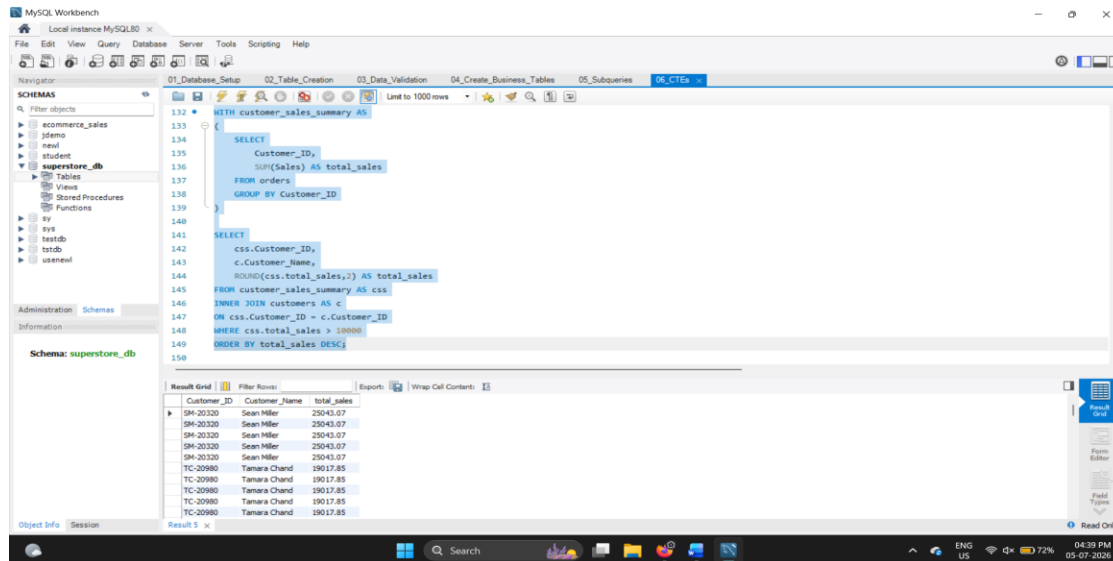


Explanation: Mirrors the total-sales CTE pattern but aggregates Profit instead, giving a profitability view of the same customer base.

Query 5 — Customers with Sales Exceeding ₹10,000

SQL Query:

```
WITH customer_sales_summary AS
(
    SELECT
        Customer_ID,
        SUM(Sales) AS total_sales
    FROM orders
    GROUP BY Customer_ID
)
SELECT
    css.Customer_ID,
    c.Customer_Name,
    ROUND(css.total_sales,2) AS total_sales
FROM customer_sales_summary AS css
INNER JOIN customers AS c
ON css.Customer_ID = c.Customer_ID
WHERE css.total_sales > 10000
ORDER BY total_sales DESC;
```



Explanation: A fixed business threshold (₹10,000) is applied directly in the WHERE clause on top of the same reusable CTE, demonstrating how the same summary can support several different filtering rules.

Query 6 — Customer Category by Total Sales

SQL Query:

```
WITH customer_sales_summary AS
(
    SELECT
        Customer_ID,
        SUM(Sales) AS total_sales
    FROM orders
    GROUP BY Customer_ID
)

SELECT
    css.Customer_ID,
    c.Customer_Name,
    ROUND(css.total_sales,2) AS total_sales,
    CASE
```

```

        WHEN css.total_sales >= 15000 THEN 'Platinum Customer'

        WHEN css.total_sales >= 10000 THEN 'Gold Customer'

        WHEN css.total_sales >= 5000 THEN 'Silver Customer'

        ELSE 'Regular Customer'

    END AS customer_category

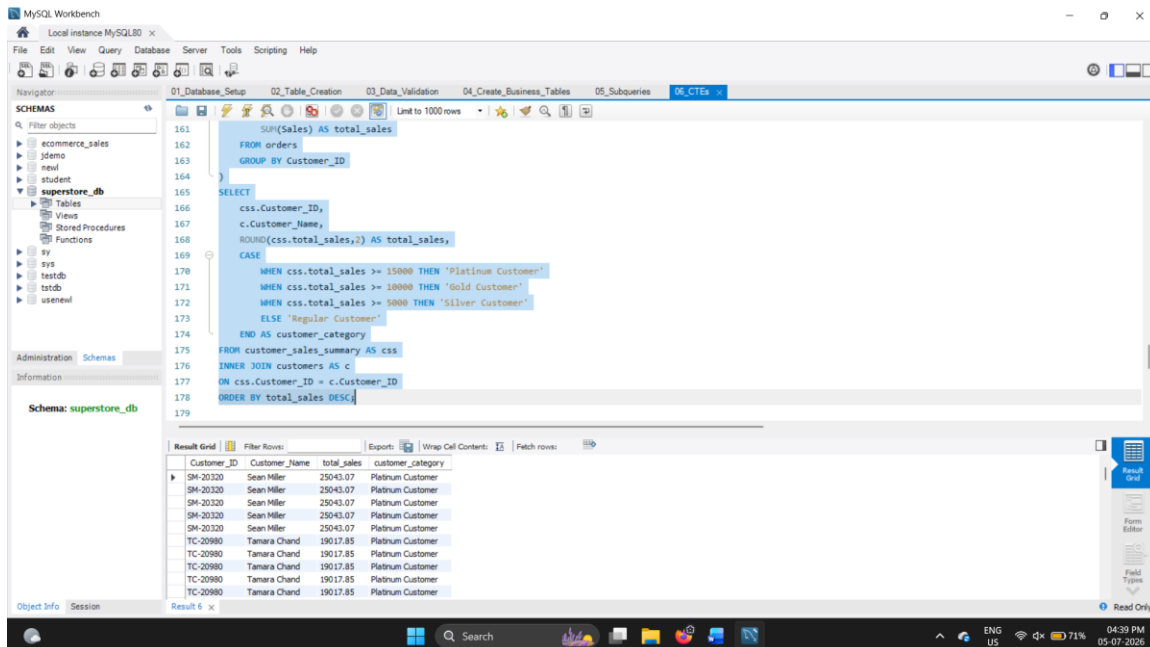
FROM customer_sales_summary AS css

INNER JOIN customers AS c

ON css.Customer_ID = c.Customer_ID

ORDER BY total_sales DESC;

```



Explanation: A CASE expression layered on top of the CTE turns a continuous sales figure into a readable loyalty tier (Platinum / Gold / Silver / Regular), which is far more useful for a business audience than the raw number alone.

Query 7 — Region-wise Customer Sales

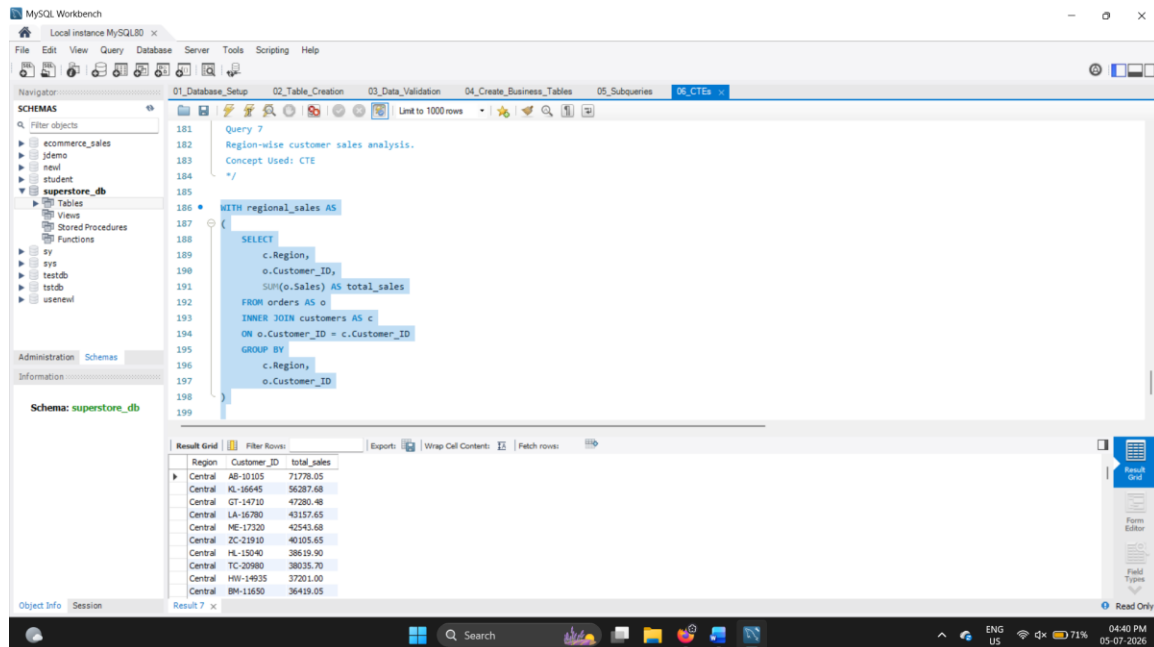
SQL Query:

```

WITH regional_sales AS

```

```
(  
    SELECT  
        c.Region,  
        o.Customer_ID,  
        SUM(o.Sales) AS total_sales  
    FROM orders AS o  
    INNER JOIN customers AS c  
    ON o.Customer_ID = c.Customer_ID  
    GROUP BY  
        c.Region,  
        o.Customer_ID  
)  
SELECT  
    Region,  
    Customer_ID,  
    ROUND(total_sales,2) AS total_sales  
FROM regional_sales  
ORDER BY  
    Region,  
    total_sales DESC;
```

Explanation: The CTE groups by both Region and Customer_ID, giving per-customer totals that stay tagged with their region, which the outer query then sorts geographically and then by spend, making it easy to see the top spenders within each region.

13. Window Functions

Window functions were used to perform ranking, sequencing, and running-total calculations without collapsing the result set the way GROUP BY does. Ten queries were written to cover the main window function families: ranking functions, ROW_NUMBER, LAG, NTILE, and windowed aggregates.

Query 1 — Rank Customers by Total Sales

Concept Applied: RANK()

SQL Query:

```

WITH customer_sales_summary AS

(

    SELECT

        Customer_ID,

        SUM(Sales) AS total_sales

    FROM orders

```

```

GROUP BY Customer_ID
)

SELECT

css.Customer_ID,

c.Customer_Name,

ROUND(css.total_sales,2) AS total_sales,

RANK() OVER(ORDER BY css.total_sales DESC) AS customer_rank

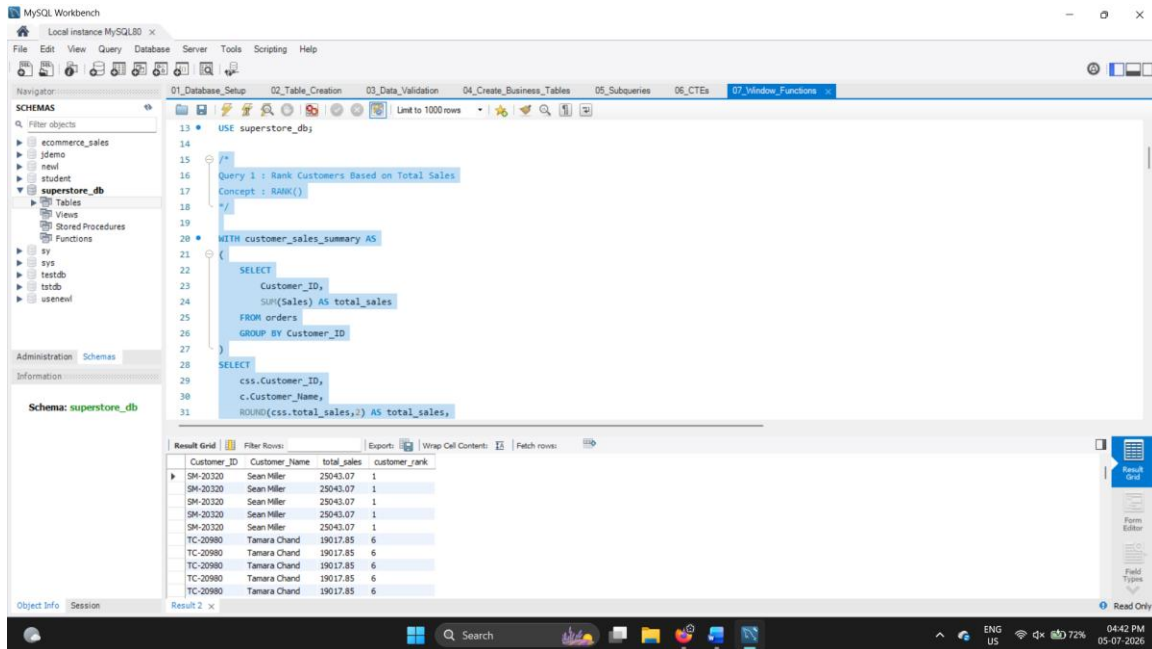
FROM customer_sales_summary AS css

INNER JOIN customers AS c

ON css.Customer_ID=c.Customer_ID

ORDER BY customer_rank;

```



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that uses a CTE to calculate total sales per customer and then ranks them. The query is as follows:

```

USE superstore_db;

/*
Query 1 : Rank Customers Based on Total Sales
Concept : RANK()
*/

WITH customer_sales_summary AS
(
SELECT
Customer_ID,
SUM(Sales) AS total_sales
FROM orders
GROUP BY Customer_ID
)

SELECT
css.Customer_ID,
c.Customer_Name,
ROUND(css.total_sales,2) AS total_sales,
RANK() OVER(ORDER BY css.total_sales DESC) AS customer_rank
FROM customer_sales_summary AS css
INNER JOIN customers AS c
ON css.Customer_ID=c.Customer_ID
ORDER BY customer_rank;

```

The Results Grid shows the output of the query:

Customer_ID	Customer_Name	total_sales	customer_rank
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
TC-20980	Tamara Chand	19017.85	6
TC-20980	Tamara Chand	19017.85	6
TC-20980	Tamara Chand	19017.85	6
TC-20980	Tamara Chand	19017.85	6

Explanation: RANK() assigns the same rank to tied sales totals and then skips the following rank number(s), which correctly reflects ties in a leaderboard-style report.

Query 2 — Dense Rank Customers

Concept Applied: DENSE_RANK()

SQL Query:

```

WITH customer_sales_summary AS
(
    SELECT Customer_ID, SUM(Sales) AS total_sales
    FROM orders GROUP BY Customer_ID
)
SELECT
    css.Customer_ID,
    c.Customer_Name,
    ROUND(css.total_sales,2) AS total_sales,
    DENSE_RANK() OVER(ORDER BY css.total_sales DESC) AS dense_rank
FROM customer_sales_summary AS css
INNER JOIN customers AS c
ON css.Customer_ID=c.Customer_ID
ORDER BY dense_rank;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

-- Concept : DENSE_RANK()
--/
WITH customer_sales_summary AS
(
    SELECT
        Customer_ID,
        SUM(Sales) AS total_sales
    FROM orders
    GROUP BY Customer_ID
)
SELECT
    css.Customer_ID,
    c.Customer_Name,
    ROUND(css.total_sales,2) AS total_sales,
    DENSE_RANK() OVER (ORDER BY css.total_sales DESC) AS sales_rank
FROM customer_sales_summary AS css
INNER JOIN customers AS c
ON css.Customer_ID = c.Customer_ID
ORDER BY sales_rank;

```

The Results Grid shows the following data:

Customer_ID	Customer_Name	total_sales	sales_rank
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
SM-20320	Sean Miller	25043.07	1
TTC-20980	Tamara Chand	19017.85	2
TTC-20980	Tamara Chand	19017.85	2
TTC-20980	Tamara Chand	19017.85	2

Explanation: DENSE_RANK() behaves like RANK() for tied values but does not leave a gap in the ranking sequence afterwards, which is the key difference between the two functions.

Query 3 — Row Number per Customer's Orders

Concept Applied: ROW_NUMBER() with PARTITION BY

SQL Query:

```
SELECT

    Order_ID,

    Customer_ID,

    Sales,

    ROW_NUMBER() OVER(

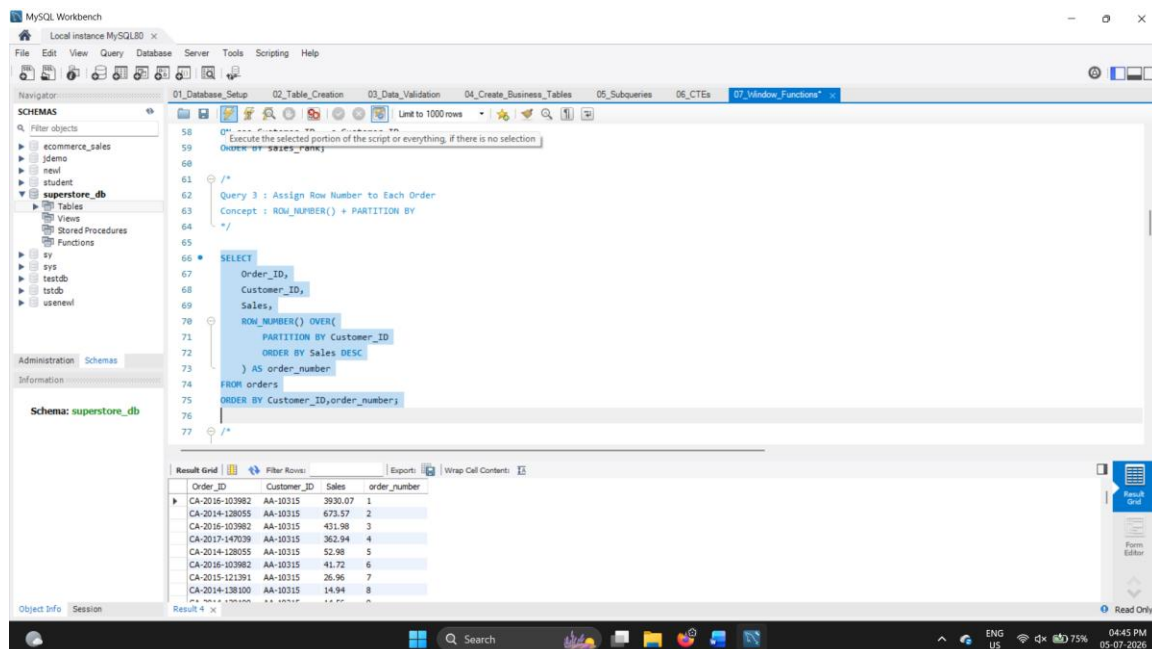
        PARTITION BY Customer_ID

        ORDER BY Sales DESC

    ) AS order_number

FROM orders

ORDER BY Customer_ID, order_number;
```



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' panel with a tree view of the 'superstore_db' database, including tables like 'ecommerce_sales', 'jdemo', 'new', 'student', 'sys', 'testdb', 'tstdb', and 'useneil'. The main editor window shows the SQL query for Query 3, which uses ROW_NUMBER() with PARTITION BY Customer_ID and ORDER BY Sales DESC to assign row numbers to orders. The bottom panel shows the 'Result Grid' with 8 rows of data, including Order_ID, Customer_ID, Sales, and order_number.

Order_ID	Customer_ID	Sales	order_number
CA-2016-103982	AA-10315	3930.07	1
CA-2014-128055	AA-10315	673.57	2
CA-2015-103982	AA-10315	431.98	3
CA-2017-147039	AA-10315	362.94	4
CA-2014-128055	AA-10315	52.88	5
CA-2016-103982	AA-10315	41.72	6
CA-2015-121391	AA-10315	26.96	7
CA-2014-138100	AA-10315	14.94	8

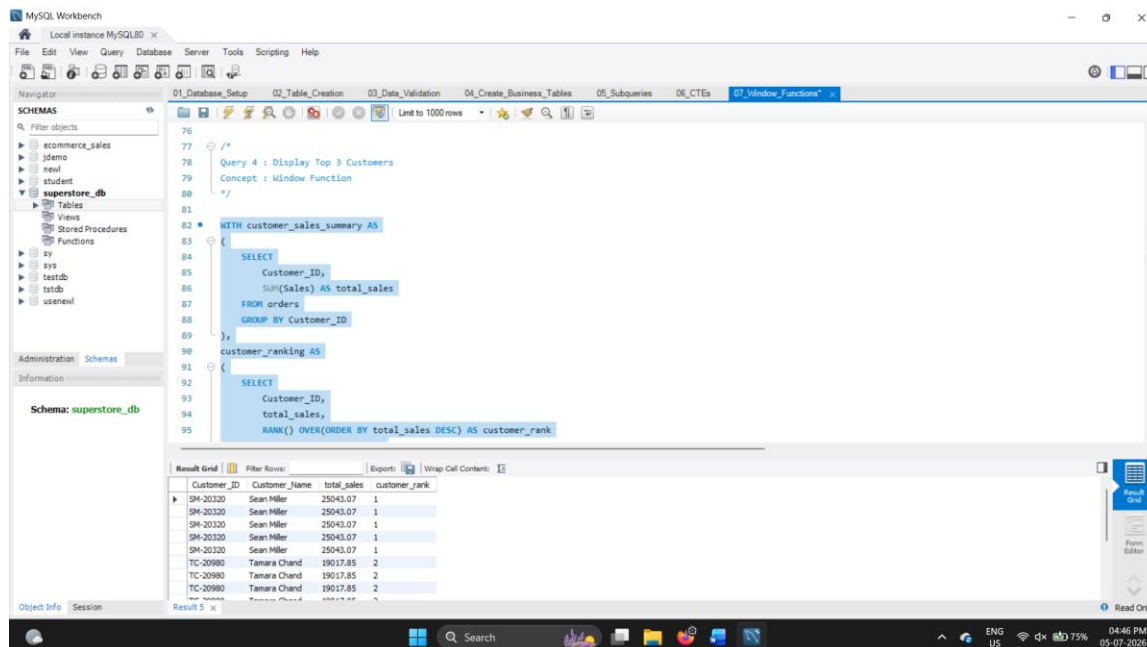
Explanation: PARTITION BY resets the numbering for every customer, so order_number = 1 always refers to that specific customer's largest order rather than a global rank.

Query 4 — Top 3 Customers

Concept Applied: Window Function + CTE chaining

SQL Query:

```
WITH customer_sales_summary AS
(
    SELECT Customer_ID, SUM(Sales) AS total_sales
    FROM orders GROUP BY Customer_ID
),
customer_ranking AS
(
    SELECT
        Customer_ID,
        total_sales,
        RANK() OVER(ORDER BY total_sales DESC) AS customer_rank
    FROM customer_sales_summary
)
SELECT
    cr.Customer_ID,
    c.Customer_Name,
    ROUND(cr.total_sales,2) AS total_sales,
    cr.customer_rank
FROM customer_ranking AS cr
INNER JOIN customers AS c
ON cr.Customer_ID=c.Customer_ID
WHERE cr.customer_rank<=3
ORDER BY cr.customer_rank;
```



Explanation: Two CTEs are chained together — the first aggregates sales, the second ranks them — and the outer query filters on the rank column. Filtering directly on a window function's alias in the WHERE clause is not permitted in the same SELECT, which is exactly why the ranking step needs its own CTE.

Query 5 — Highest Sale per Customer (via ROW_NUMBER)

Concept Applied: ROW_NUMBER()

SQL Query:

```

WITH ranked_orders AS
(
    SELECT
        Order_ID, Customer_ID, Sales, Profit,
        ROW_NUMBER() OVER(
            PARTITION BY Customer_ID
            ORDER BY Sales DESC
        ) AS row_num
    FROM orders
)

```

```
SELECT
```

```
    ro.Order_ID, ro.Customer_ID, c.Customer_Name, ro.Sales, ro.Profit
```

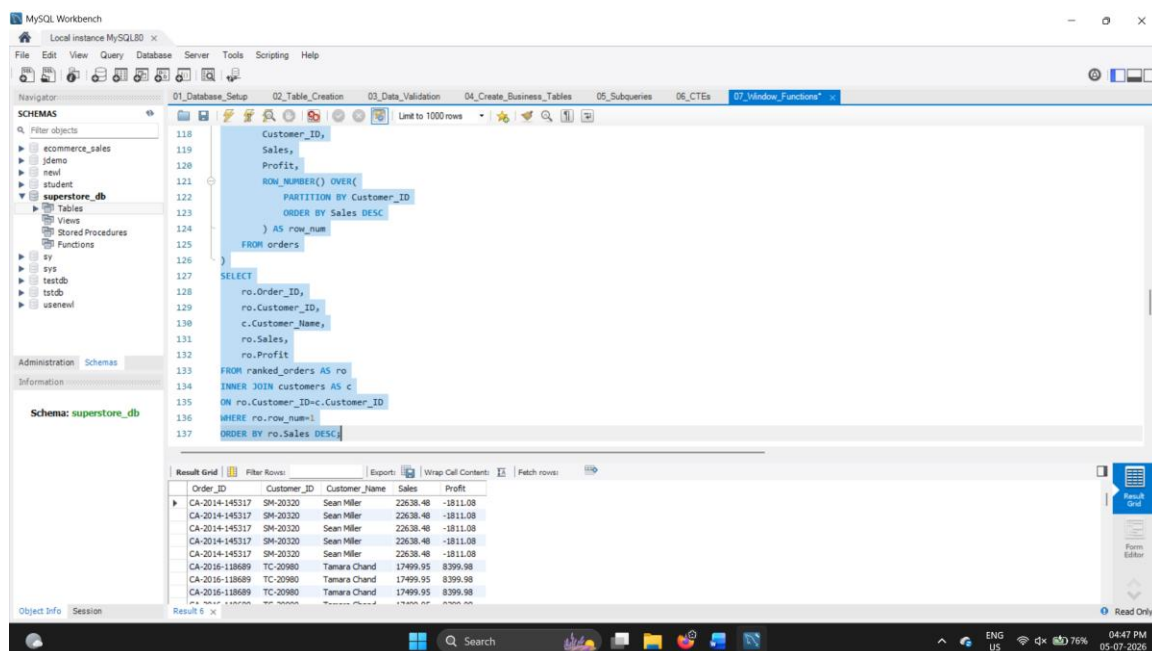
```
FROM ranked_orders AS ro
```

```
INNER JOIN customers AS c
```

```
ON ro.Customer_ID=c.Customer_ID
```

```
WHERE ro.row_num=1
```

```
ORDER BY ro.Sales DESC;
```



Explanation: This achieves the same business outcome as the correlated subquery in Section 11, Query 2 (each customer's largest order), but does it with a single pass using ROW_NUMBER() instead of re-running MAX() once per customer — generally the more efficient approach on larger tables.

Query 6 — Previous Order's Sales Value

Concept Applied: LAG()

SQL Query:

```
SELECT
```

```
    Customer_ID, Order_ID, Sales,
```

```

LAG(Sales) OVER(

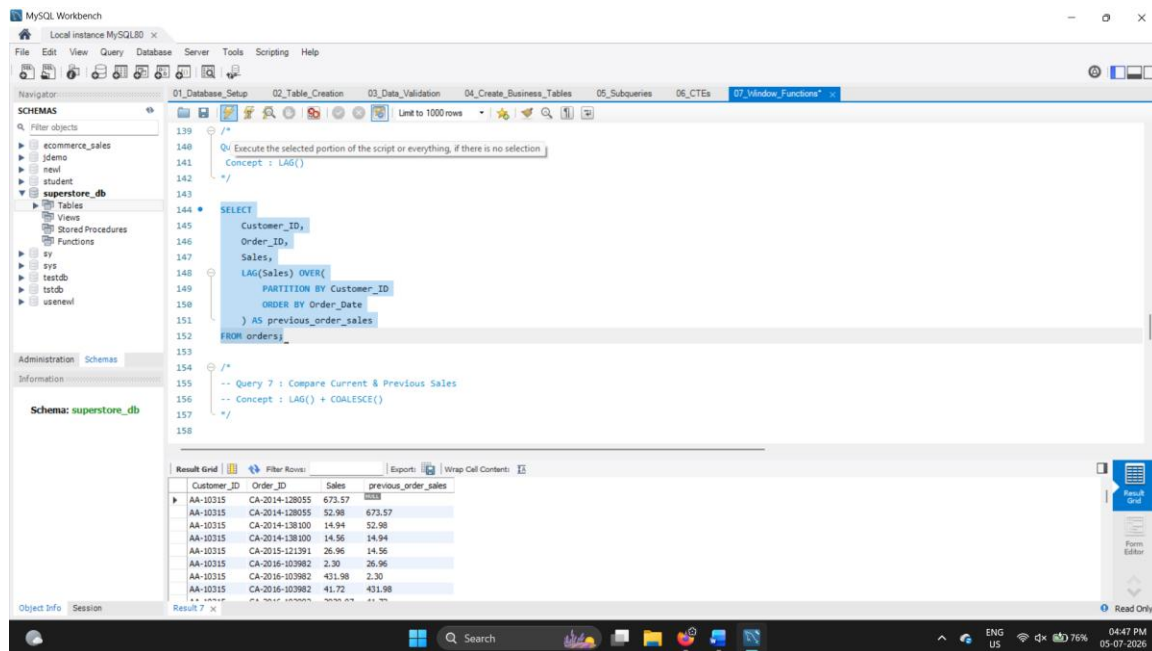
    PARTITION BY Customer_ID

    ORDER BY Order_Date

) AS previous_order_sales

FROM orders;

```



Explanation: LAG() looks one row back within each customer's own order history (ordered by date), making it possible to compare a customer's current order against their previous one without a self-join.

Query 7 — Current vs Previous Sales (with COALESCE)

Concept Applied: LAG() + COALESCE()

SQL Query:

```

SELECT

    Customer_ID, Order_ID, Sales,

    COALESCE(

        LAG(Sales) OVER(

```

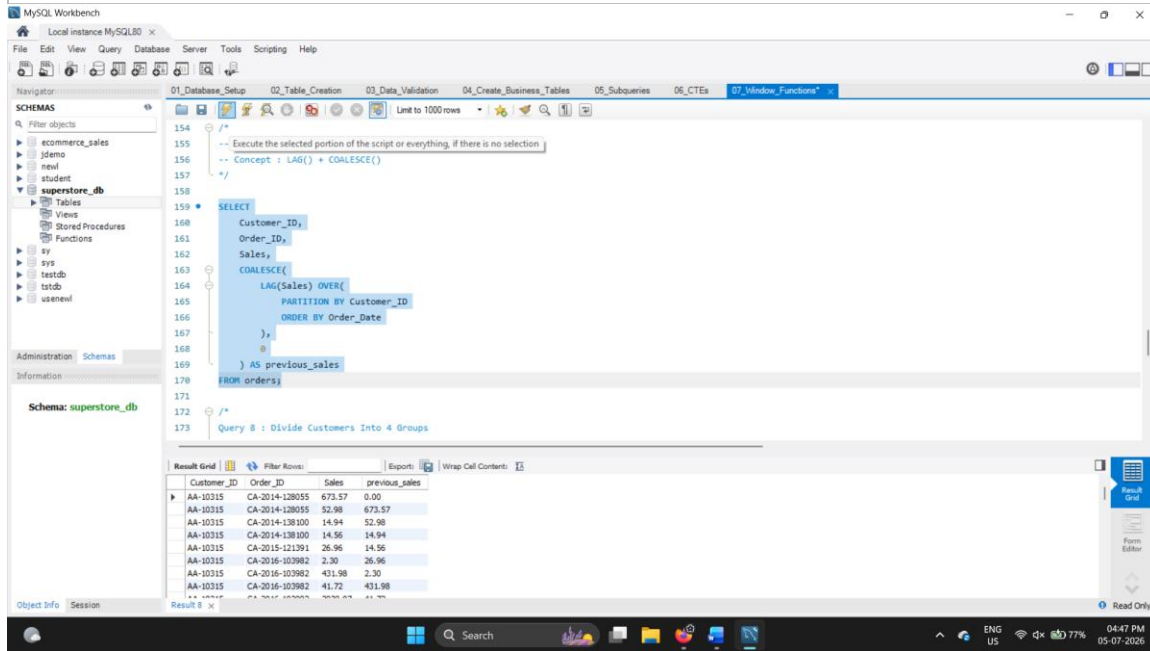

PARTITION BY Customer_ID

ORDER BY Order_Date

), 0

) AS previous_sales

FROM orders;



Explanation: Builds directly on Query 6: since a customer's very first order has no prior row, LAG() would return NULL there. COALESCE() substitutes 0 in that case so the column is always numeric and safe to use in further calculations.

Query 8 — Divide Customers into 4 Spend Quartiles

Concept Applied: NTILE()

SQL Query:

WITH customer_sales_summary AS

(

SELECT Customer_ID, SUM(Sales) AS total_sales

FROM orders GROUP BY Customer_ID

```

)

SELECT

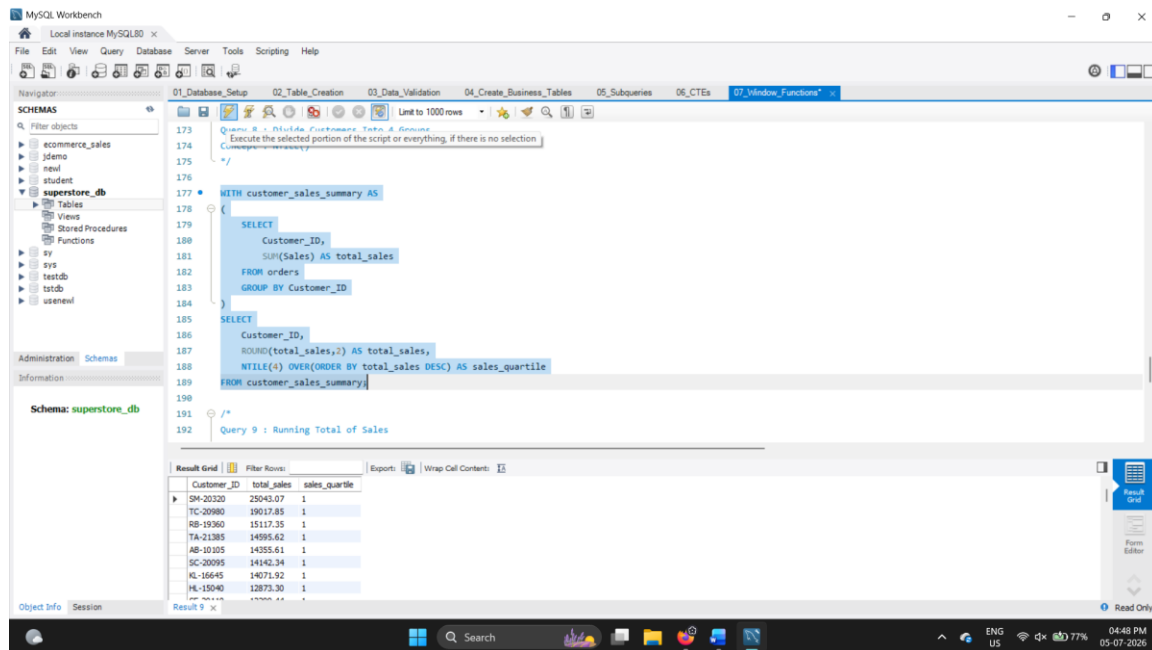
    Customer_ID,

    ROUND(total_sales,2) AS total_sales,

    NTILE(4) OVER(ORDER BY total_sales DESC) AS sales_quartile

FROM customer_sales_summary;

```



Explanation: NTILE(4) divides the ranked customer list into four roughly equal-sized buckets, giving a quick quartile segmentation (top 25%, next 25%, and so on) without needing to define fixed sales thresholds by hand.

Query 9 — Running Total of Sales

Concept Applied: SUM() OVER() - Running Total

SQL Query:

```

SELECT

    Order_Date, Order_ID, Sales,

    SUM(Sales) OVER(

```

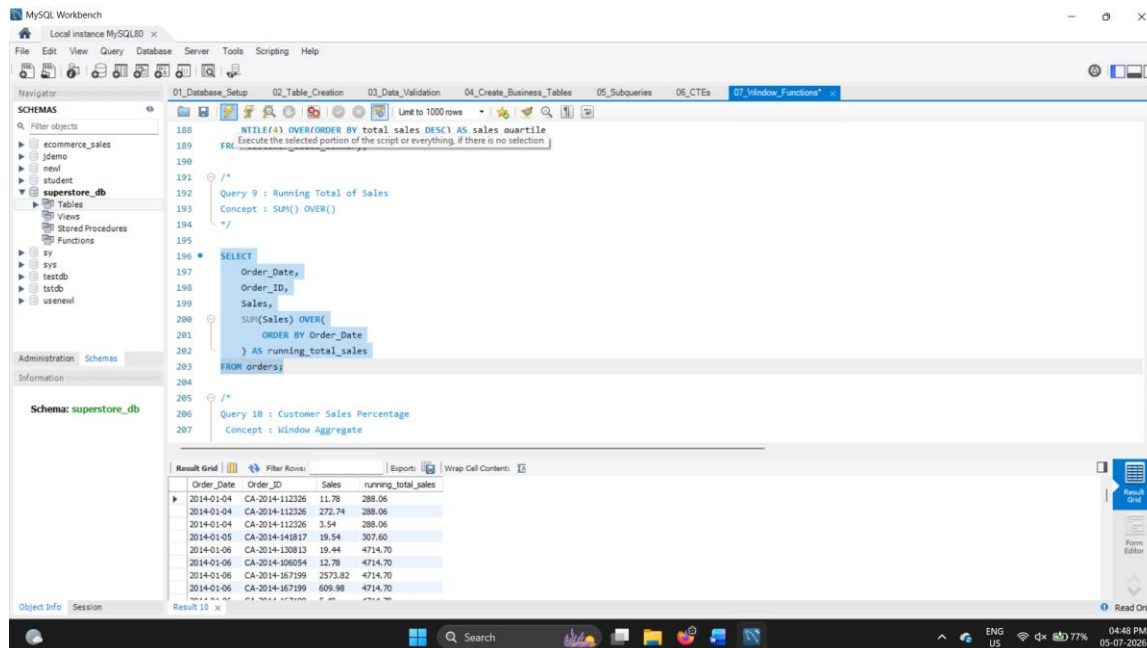
```

ORDER BY Order_Date

) AS running_total_sales

FROM orders;

```



Explanation: With no PARTITION BY, the running total accumulates across the entire orders table in date order, showing cumulative revenue growth over time.

Query 10 — Customer Sales Percentage of Total

Concept Applied: Window Aggregate

SQL Query:

```

WITH customer_sales_summary AS

(

    SELECT Customer_ID, SUM(Sales) AS total_sales

    FROM orders GROUP BY Customer_ID

)

SELECT

    Customer_ID,

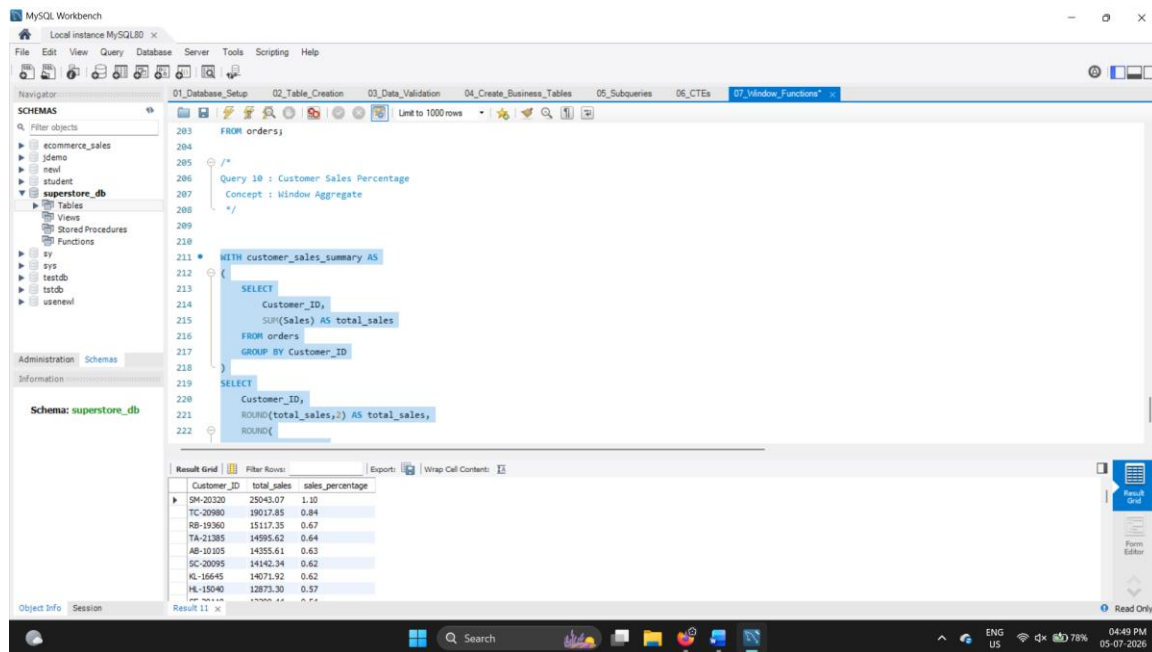
```

```
ROUND(total_sales,2) AS total_sales,
```

```
ROUND(total_sales / SUM(total_sales) OVER() * 100, 2) AS sales_percentage
```

```
FROM customer_sales_summary
```

```
ORDER BY total_sales DESC;
```



Explanation: SUM(total_sales) OVER() with no ORDER BY or PARTITION BY computes the grand total across all rows while keeping each individual customer's row intact, allowing each customer's share of total company sales to be calculated in the same query.

14. Final Combined Query:

The final stage of the analytical build brought joins, CTEs, and window functions together into a single consolidated customer report, produced in three increasingly detailed versions.

Version 1 — Basic Ranked Report

SQL Query:

```
WITH customer_sales AS
```

```
(
```

```

SELECT o.Customer_ID, SUM(o.Sales) AS total_sales

FROM orders AS o

GROUP BY o.Customer_ID
)

SELECT

    c.Customer_ID,

    c.Customer_Name,

    ROUND(cs.total_sales,2) AS total_sales,

    RANK() OVER(ORDER BY cs.total_sales DESC) AS customer_rank

FROM customer_sales AS cs

INNER JOIN customers AS c

ON cs.Customer_ID=c.Customer_ID

ORDER BY customer_rank;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

-- Final Combined Query
WITH customer_sales AS
(
    SELECT
        o.Customer_ID,
        SUM(o.Sales) AS total_sales
    FROM orders AS o
    GROUP BY o.Customer_ID
)
SELECT
    c.Customer_ID,
    c.Customer_Name,
    ROUND(cs.total_sales,2) AS total_sales,
    RANK() OVER(ORDER BY cs.total_sales DESC) AS customer_rank
FROM customer_sales AS cs
INNER JOIN customers AS c
ON cs.Customer_ID=c.Customer_ID
ORDER BY customer_rank;

```

The Results Grid shows the following data:

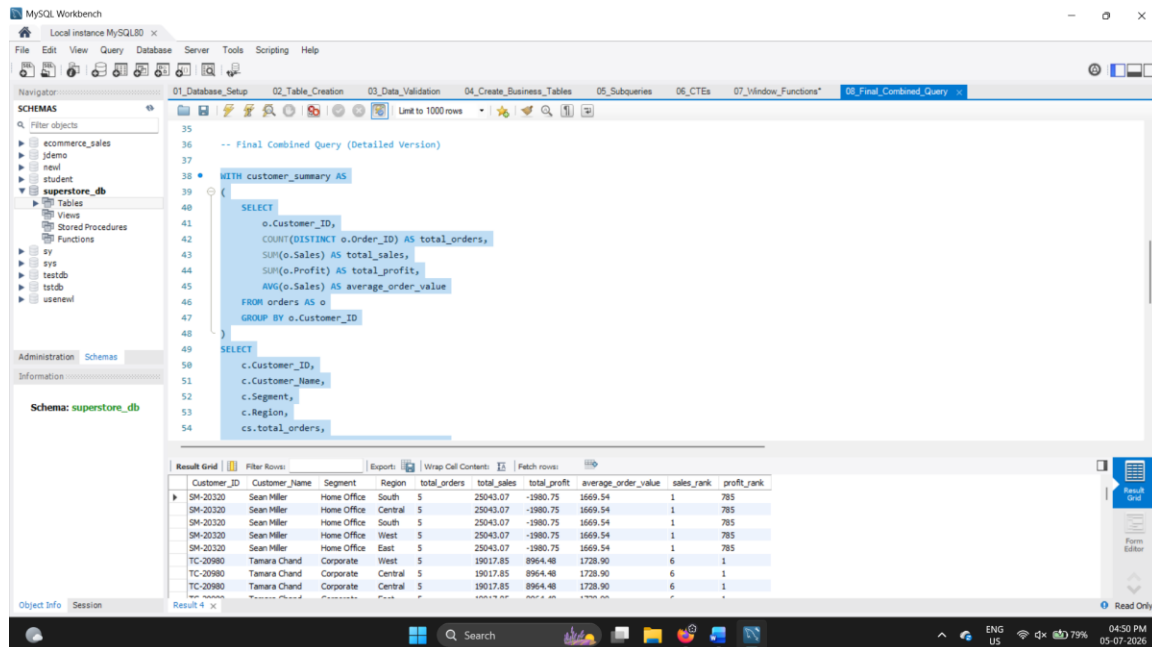
Customer_ID	Customer_Name	total_sales	customer_rank
SH-20320	Sean Miller	25043.07	1
SH-20320	Sean Miller	25043.07	1
SH-20320	Sean Miller	25043.07	1
SH-20320	Sean Miller	25043.07	1
SH-20320	Sean Miller	25043.07	1
SH-20320	Sean Miller	25043.07	1
TC-20980	Tamara Chand	19017.85	6
TC-20980	Tamara Chand	19017.85	6
TC-20980	Tamara Chand	19017.85	6

Explanation: A clean, minimal customer ranking report — total sales joined with the customer name and given a rank.

Version 2 — Detailed Report

SQL Query:

```
WITH customer_summary AS
(
    SELECT
        o.Customer_ID,
        COUNT(DISTINCT o.Order_ID) AS total_orders,
        SUM(o.Sales) AS total_sales,
        SUM(o.Profit) AS total_profit,
        AVG(o.Sales) AS average_order_value
    FROM orders AS o
    GROUP BY o.Customer_ID
)
SELECT
    c.Customer_ID, c.Customer_Name, c.Segment, c.Region,
    cs.total_orders,
    ROUND(cs.total_sales,2) AS total_sales,
    ROUND(cs.total_profit,2) AS total_profit,
    ROUND(cs.average_order_value,2) AS average_order_value,
    RANK() OVER(ORDER BY cs.total_sales DESC) AS sales_rank,
    DENSE_RANK() OVER(ORDER BY cs.total_profit DESC) AS profit_rank
FROM customer_summary AS cs
INNER JOIN customers AS c
ON cs.Customer_ID=c.Customer_ID
ORDER BY sales_rank;
```



Explanation: Extends the basic report with segment and region context plus two independent rankings — by sales and by profit — calculated in the same result set, which makes it easy to spot customers who rank highly on one metric but not the other.

Version 3 — Top 10 Customer Report

SQL Query:

```
WITH customer_summary AS
(
    SELECT Customer_ID, SUM(Sales) AS total_sales
    FROM orders GROUP BY Customer_ID
),
customer_ranking AS
(
    SELECT
        Customer_ID, total_sales,
        RANK() OVER(ORDER BY total_sales DESC) AS customer_rank
    FROM customer_summary
)
```

```

SELECT

    cr.customer_rank,

    c.Customer_Name,

    ROUND(cr.total_sales,2) AS total_sales

FROM customer_ranking AS cr

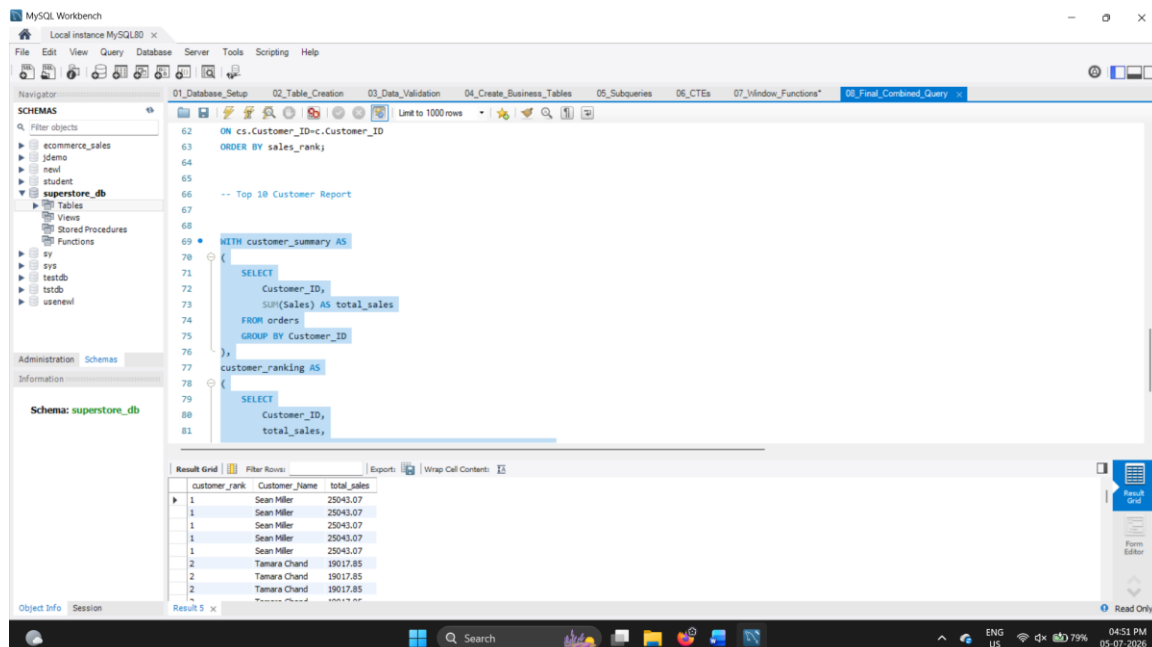
INNER JOIN customers AS c

ON cr.Customer_ID=c.Customer_ID

WHERE cr.customer_rank<=10

ORDER BY cr.customer_rank;

```



Explanation: A condensed, presentation-ready leaderboard of the ten highest-spending customers, produced by filtering the ranked CTE down to rank 10 or better.

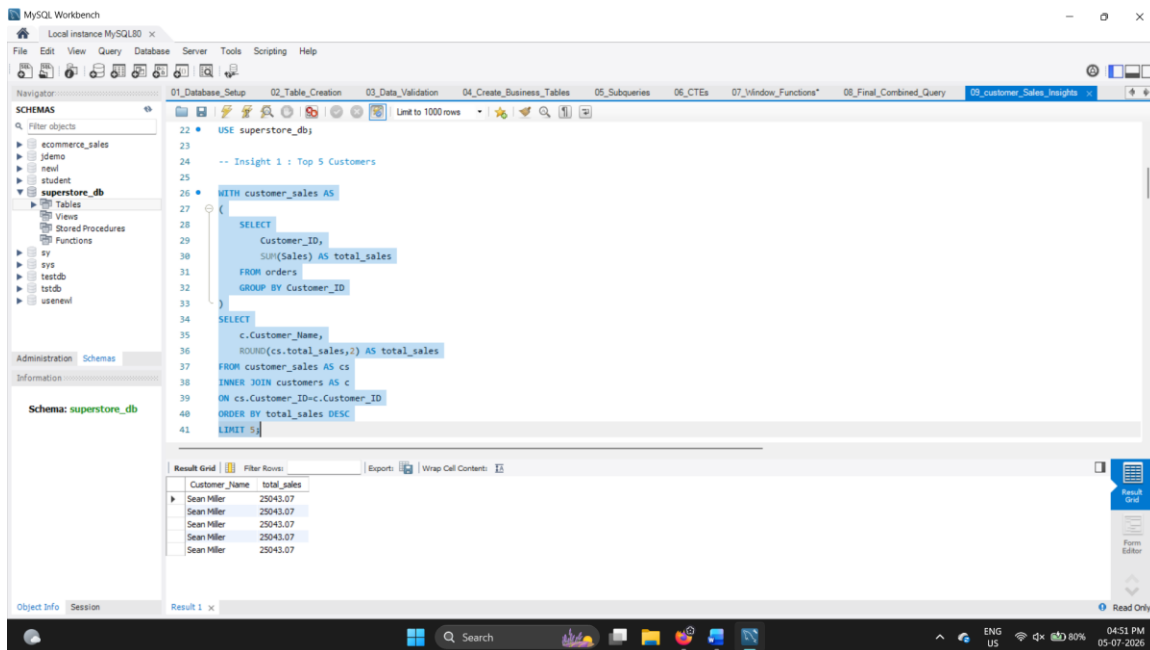
15. Customer Sales Insights (Mini Project) :

The final script of the assignment was framed as a mini-project answering ten specific business questions about customer behaviour, with five extension insights added afterwards to broaden the analysis to profitability, regions, and segments.

Insight 1 — Top 5 Customers

SQL Query:

```
WITH customer_sales AS
(
    SELECT Customer_ID, SUM(Sales) AS total_sales
    FROM orders GROUP BY Customer_ID
)
SELECT c.Customer_Name, ROUND(cs.total_sales,2) AS total_sales
FROM customer_sales AS cs
INNER JOIN customers AS c ON cs.Customer_ID=c.Customer_ID
ORDER BY total_sales DESC
LIMIT 5;
```



Explanation: Answers the first mini-project question directly — the five customers who have generated the most total sales value.

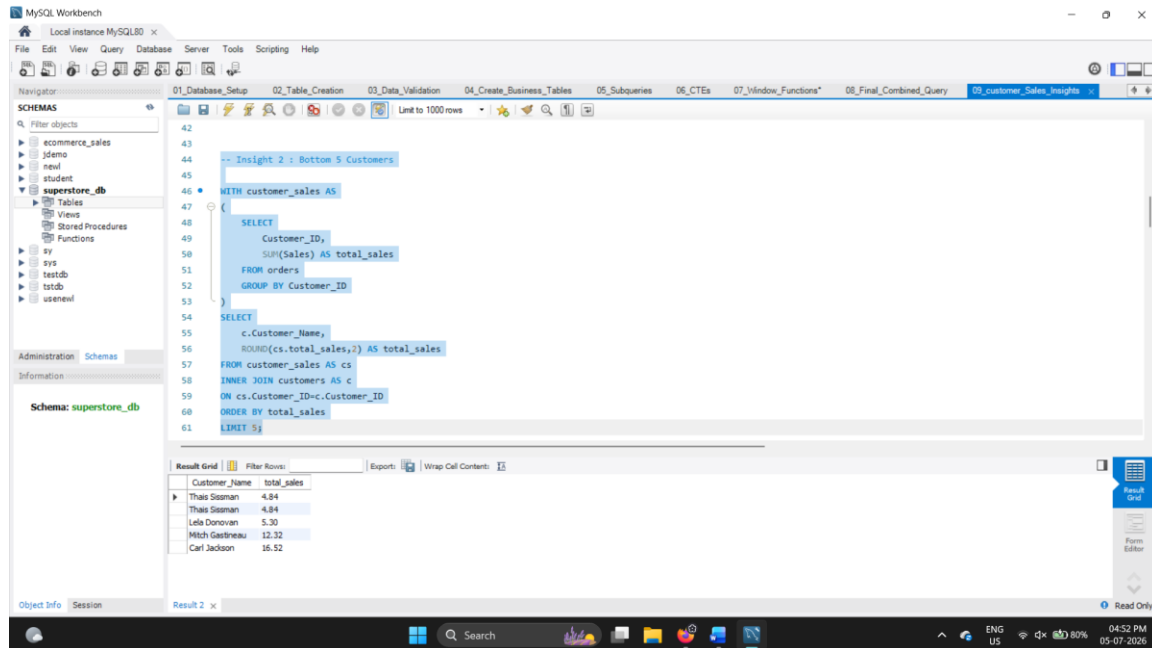
Insight 2 — Bottom 5 Customers

SQL Query:

-- Same CTE as Insight 1, sorted ascending

ORDER BY total_sales

LIMIT 5;



Explanation: The mirror image of Insight 1: removing DESC from the ORDER BY (and keeping the same LIMIT 5) surfaces the five lowest-spending customers instead, useful for identifying accounts that may need re-engagement.

Insight 3 — Customers with Exactly One Order

SQL Query:

```
SELECT

    c.Customer_Name,

    COUNT(o.Order_ID) AS total_orders

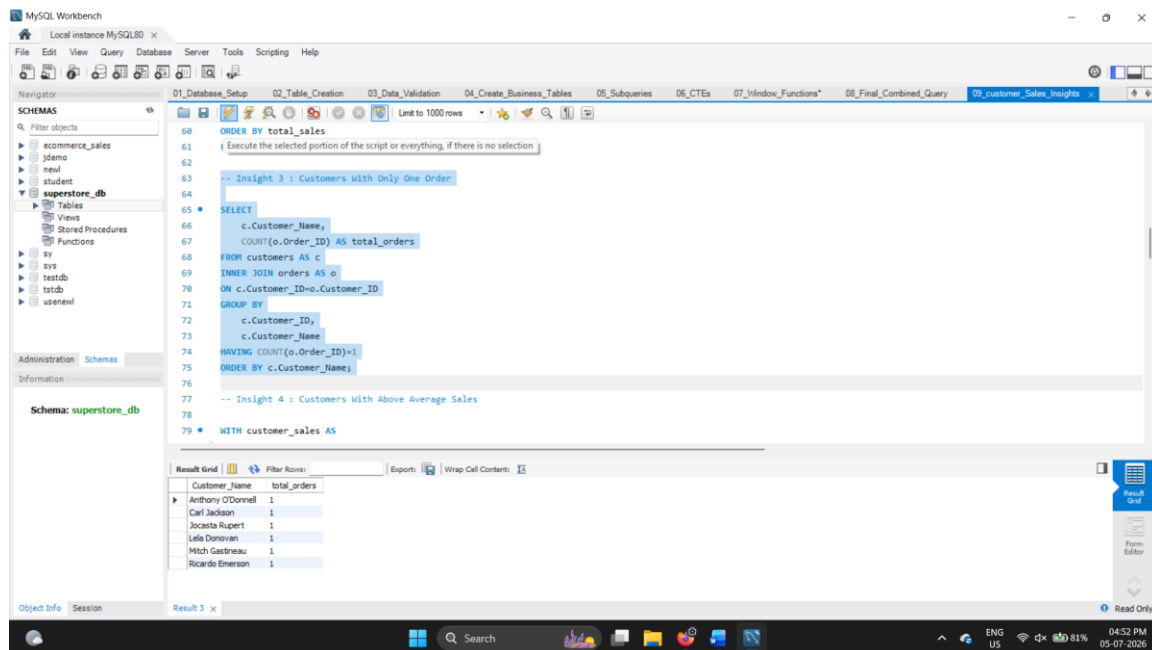
FROM customers AS c

INNER JOIN orders AS o ON c.Customer_ID=o.Customer_ID

GROUP BY c.Customer_ID, c.Customer_Name

HAVING COUNT(o.Order_ID)=1
```

```
ORDER BY c.Customer_Name;
```



Explanation: HAVING COUNT(...) = 1 filters the grouped results down to customers who have never placed more than one order — a useful segment for one-time-buyer retention campaigns.

Insight 4 — Customers with Above-Average Sales

SQL Query:

```
WITH customer_sales AS

(

    SELECT Customer_ID, SUM(Sales) AS total_sales

    FROM orders GROUP BY Customer_ID

)

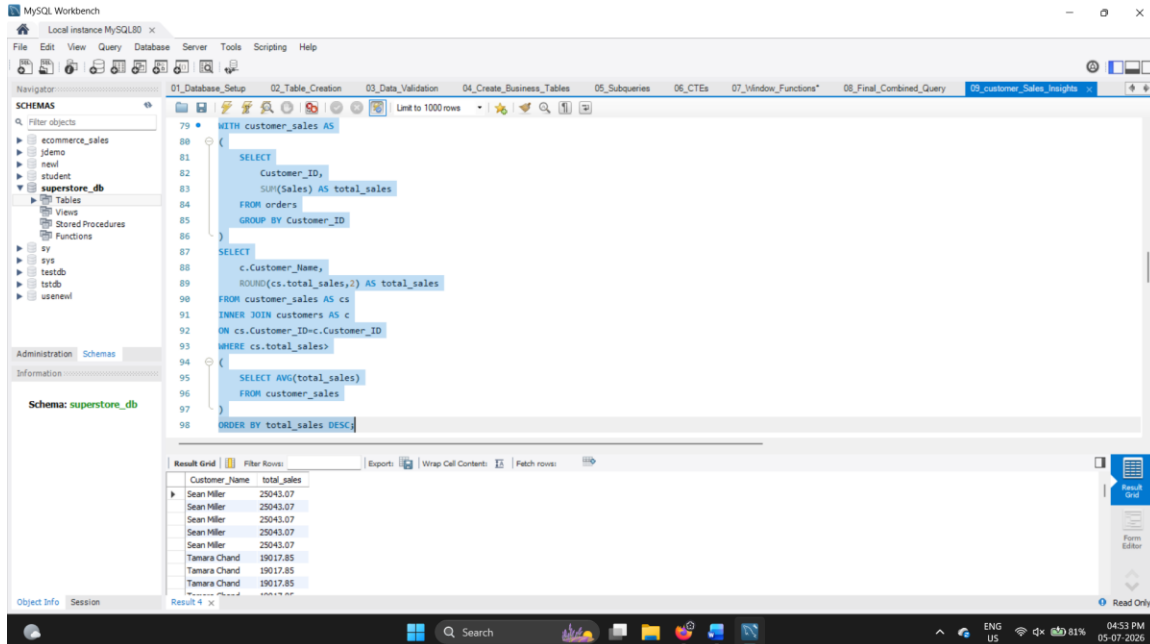
SELECT c.Customer_Name, ROUND(cs.total_sales,2) AS total_sales

FROM customer_sales AS cs

INNER JOIN customers AS c ON cs.Customer_ID=c.Customer_ID

WHERE cs.total_sales > (SELECT AVG(total_sales) FROM customer_sales)

ORDER BY total_sales DESC;
```



Explanation: Combines a CTE with a scalar subquery to isolate customers spending more than the company-wide average — the same logical pattern used in Section 12, Query 2, applied here as a direct mini-project answer.

Insight 5 — Highest Order Value per Customer

SQL Query:

```
SELECT
    c.Customer_Name, o.Order_ID, o.Sales

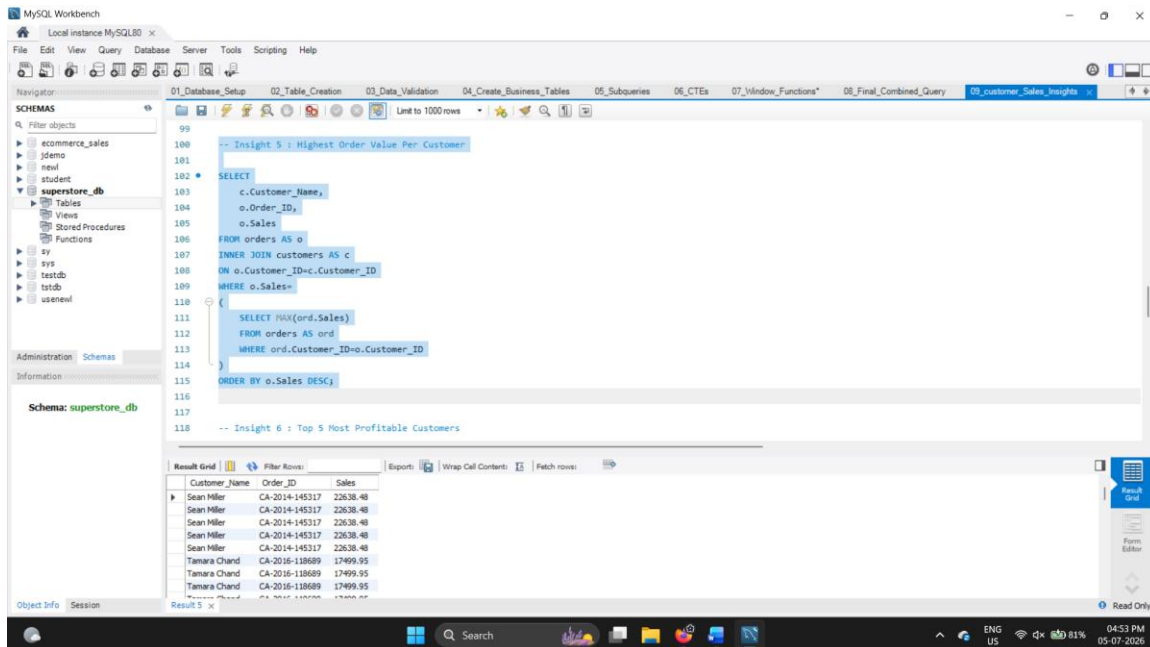
FROM orders AS o

INNER JOIN customers AS c ON o.Customer_ID=c.Customer_ID

WHERE o.Sales =

(
    SELECT MAX(ord.Sales)
    FROM orders AS ord
    WHERE ord.Customer_ID=o.Customer_ID
)

ORDER BY o.Sales DESC;
```



Explanation: A correlated subquery finds each customer's personal maximum order value, and the outer query attaches the customer's readable name to it.

Insight 6 — Top 5 Most Profitable Customers

SQL Query:

```
SELECT

  c.Customer_Name, ROUND(SUM(o.Profit),2) AS total_profit

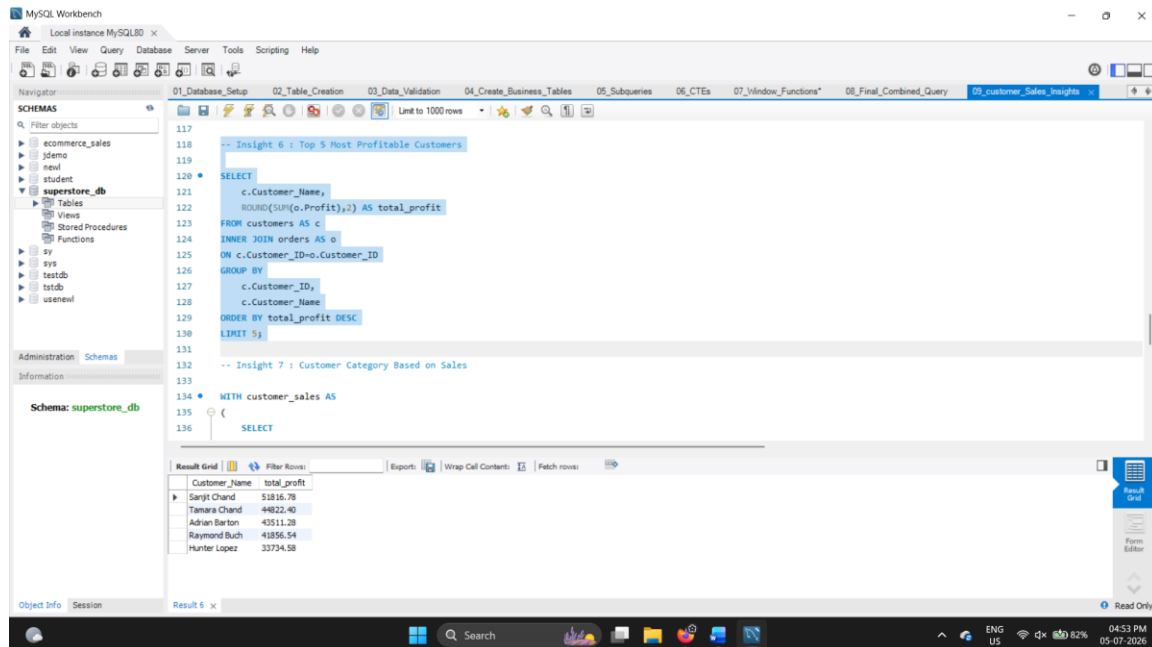
FROM customers AS c

INNER JOIN orders AS o ON c.Customer_ID=o.Customer_ID

GROUP BY c.Customer_ID, c.Customer_Name

ORDER BY total_profit DESC

LIMIT 5;
```



Explanation: Deliberately looks at Profit rather than Sales, since the two rankings do not always agree — a high-revenue customer can still be a low-profit one after discounts are taken into account.

Insight 7 — Customer Category by Sales

SQL Query:

```

WITH customer_sales AS
(
  SELECT Customer_ID, SUM(Sales) AS total_sales
  FROM orders GROUP BY Customer_ID
)
SELECT
  c.Customer_Name,
  ROUND(cs.total_sales,2) AS total_sales,
  CASE
    WHEN cs.total_sales >= 15000 THEN 'Platinum'
    WHEN cs.total_sales >= 10000 THEN 'Gold'
    WHEN cs.total_sales >= 5000 THEN 'Silver'

```

```

ELSE 'Regular'

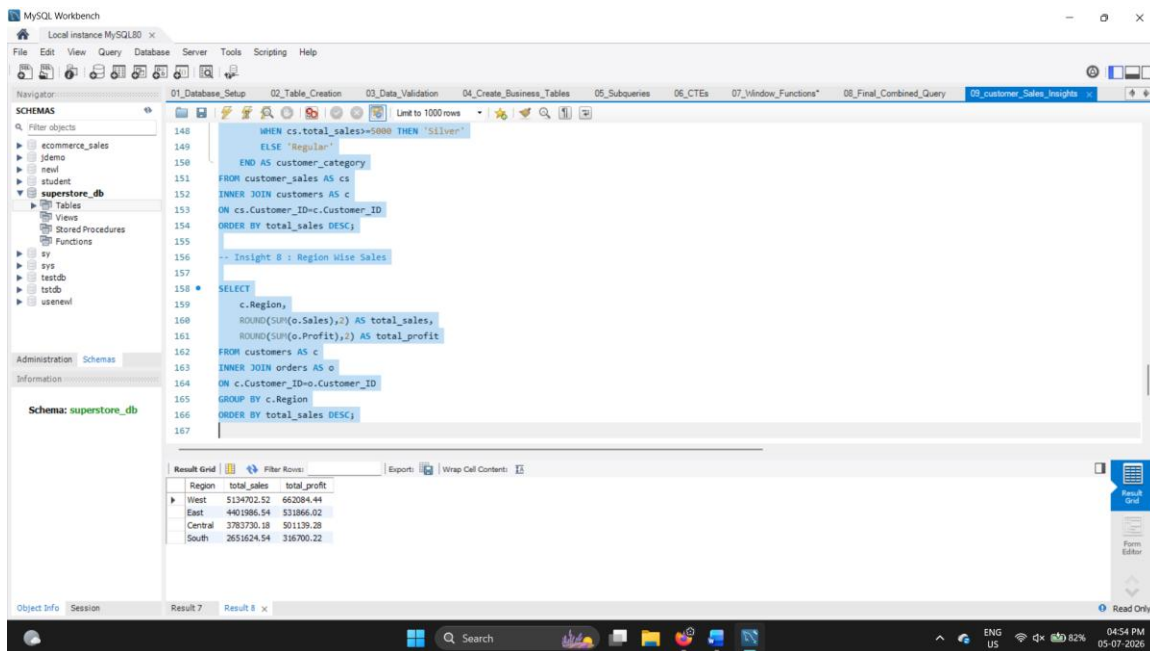
END AS customer_category

FROM customer_sales AS cs

INNER JOIN customers AS c ON cs.Customer_ID=c.Customer_ID

ORDER BY total_sales DESC;

```



Explanation: Reapplies the loyalty-tier CASE logic from the CTE section directly against the mini-project's own customer_sales CTE, keeping the insight script self-contained.

Insight 8 - Overall Business Summary

SQL Query:

```

SELECT

COUNT(DISTINCT Customer_ID) AS total_customers,

COUNT(DISTINCT Order_ID) AS total_orders,

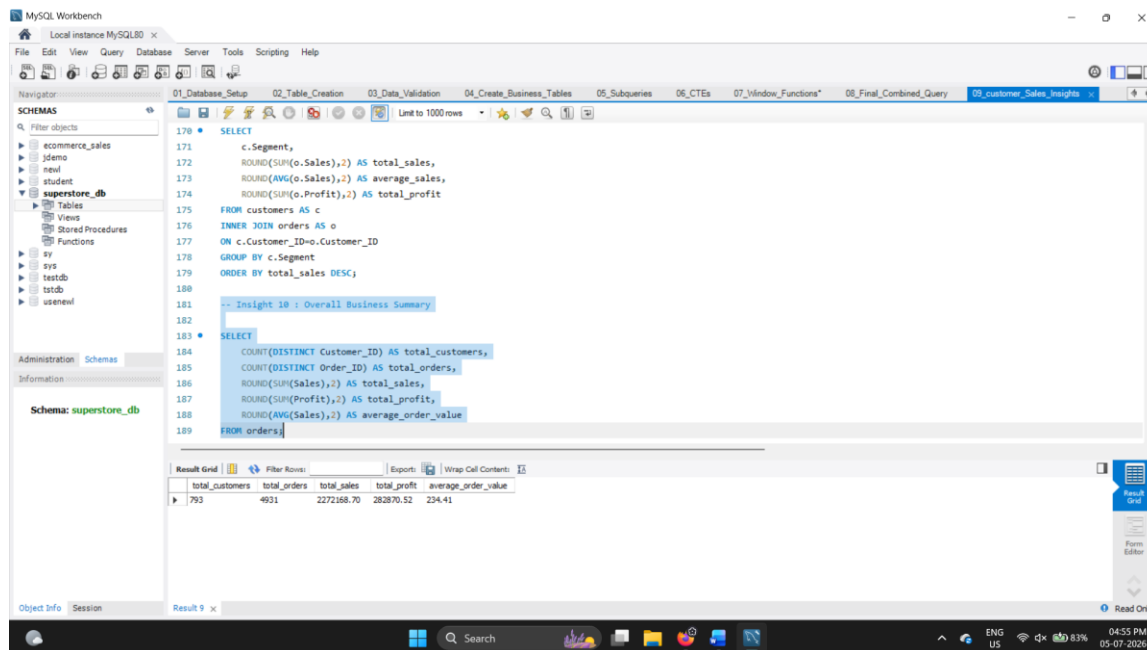
ROUND(SUM(Sales),2) AS total_sales,

ROUND(SUM(Profit),2) AS total_profit,

ROUND(AVG(Sales),2) AS average_order_value

```

FROM orders;



Explanation: A single-row executive summary of the whole orders table — the kind of headline figures that would typically open a business dashboard.

16. Performance Optimization:

- A small group of top customers accounts for a disproportionately large share of total company sales, which is why identifying and retaining them (Sections 13 and 15) matters more than broad, undifferentiated marketing.
- Sales rank and profit rank do not always move together — a customer or region that looks strong on revenue can still be weak on margin once discounts are factored in, which is exactly what the dual sales_rank / profit_rank report in Section 14 was built to reveal.
- Segmenting customers into quartiles (NTILE) or loyalty tiers (CASE-based Platinum/Gold/Silver/Regular) turns a continuous sales figure into categories a business team can act on directly, such as prioritising retention offers for the top quartile.
- One-time buyers (Insight 3) represent an identifiable segment worth investigating separately from repeat customers, since the two groups likely respond to very different retention strategies.
- Region and segment level breakdowns show that overall company totals can hide meaningfully different performance across geography and customer type.
- Data validation before analysis — checking nulls, duplicates, and row counts — is what makes every later number in this report trustworthy; skipping that step would have put every downstream ranking and total at risk of being wrong.

17. Key Business Insights:

- A small group of top customers accounts for a disproportionately large share of total company sales, which is why identifying and retaining them (Sections 13 and 15) matters more than broad, undifferentiated marketing.
- Sales rank and profit rank do not always move together — a customer or region that looks strong on revenue can still be weak on margin once discounts are factored in, which is exactly what the dual sales_rank / profit_rank report in Section 14 was built to reveal.
- Segmenting customers into quartiles (NTILE) or loyalty tiers (CASE-based Platinum/Gold/Silver/Regular) turns a continuous sales figure into categories a business team can act on directly, such as prioritising retention offers for the top quartile.
- One-time buyers (Insight 3) represent an identifiable segment worth investigating separately from repeat customers, since the two groups likely respond to very different retention strategies.
- Region and segment level breakdowns show that overall company totals can hide meaningfully different performance across geography and customer type.
- Data validation before analysis — checking nulls, duplicates, and row counts — is what makes every later number in this report trustworthy; skipping that step would have put every downstream ranking and total at risk of being wrong.

18. Challenges Faced:

- Getting the raw CSV to import cleanly required keeping Order_Date and Ship_Date as VARCHAR at first, since importing them directly as DATE caused format-mismatch errors with the source file's MM/DD/YYYY layout.
- Converting those text dates correctly with STR_TO_DATE() meant being precise about the format string (%m/%d/%Y) — a wrong format specifier silently produces NULL dates rather than an error.
- Understanding when MySQL treats a subquery as correlated versus non-correlated took some care, particularly in Section 11, since the presence of a single outer-query reference inside the WHERE clause completely changes how many times the subquery actually runs.
- Working out why a window function's alias could not be filtered directly in the same SELECT's WHERE clause (Section 13, Query 4) — the fix was wrapping the ranking logic in its own CTE and filtering one level up.
- Designing the three normalized business tables required deciding exactly which columns belonged to customers versus orders versus products, since the original flat file mixed all three together.
- Keeping the Postal_Code column as text rather than a number, to avoid losing leading zeros on certain postal codes.

19. Learning Outcomes:

- Designed a layered (bronze/silver/gold) data pipeline inside a single MySQL database.
- Practiced safe CSV import techniques and post-import data type correction.
- Applied systematic data validation covering row counts, nulls, duplicates, and statistical sanity checks.
- Normalized a flat transactional dataset into separate customer, order, and product tables.
- Wrote plain, correlated, and nested subqueries to answer comparative business questions.
- Used Common Table Expressions to simplify multi-step aggregations and avoid duplicated logic.
- Implemented ranking and analytical window functions — RANK(), DENSE_RANK(), ROW_NUMBER(), LAG(), NTILE(), and windowed SUM() — for customer segmentation and trend analysis.
- Combined joins, CTEs, and window functions into a single, presentation-ready report.
- Practiced professional SQL formatting: consistent aliasing, comments, and script organization.

20. Conclusion:

This assignment moved well beyond simple SELECT statements into the kind of layered, analytical SQL that a real data engineering or analytics role would expect. Starting from a single raw CSV file, the project built a structured pipeline — staging, validation, cleaning, and normalization — before layering subqueries, CTEs, and window functions on top of it to answer genuine customer-sales questions. The customer sales insights mini-project then tied all of that technique back to concrete business questions: who the best and weakest customers are, which regions and segments perform best, and how customer value can be segmented for further action.

Beyond the specific queries, the exercise reinforced a broader habit that is easy to skip under time pressure but is central to good analytics work: validating data before trusting any number derived from it, and structuring queries — through CTEs and normalized tables — so that logic is written once and reused rather than repeated. That combination of correctness and clarity is what this report has tried to demonstrate throughout.

21. GitHub Repository: https://github.com/snehubhosle/CSI_Weekly_Assignments

All SQL scripts referenced in this report — database setup, table creation, validation, business table creation, subqueries, CTEs, window functions, the final combined query, and the customer sales insights mini-project — are version-controlled and available in the project's GitHub repository, organized as individual, numbered .sql files corresponding to each stage described above.

